



UNIVERSITY OF TARTU



Enterprise System Integration (MTAT.03.229)

LECTURE 5: DOMAIN-DRIVEN DESIGN (DDD) APPROACH I – FUNDAMENTALS

**MOHAMAD GHARIB
UNIVERSITY OF TARTU**



Before we start

Writing in space ...

Before we start

Writing in space ...

Ordinary ball point pens did not work in space because the **ink would not flow by gravity to the ball** and the **pen leaked the required pressure**.



Before we start

Writing in space ...

Ordinary ball point pens did not work in space because the **ink would not flow by gravity to the ball** and the **pen leaked the required pressure**.



Solution: the **Fisher Space Pen** is a **gas-charged ball point pen** that is rugged and works in a wider variety of conditions, such as **zero gravity**, vacuum and extreme temperatures.

Before we start

Writing in space ...

Ordinary ball point pens did not work in space because the **ink** would not flow by gravity to the **ball** and the **pen** leaked the required pressure.



Before we start

Writing in space ...

Example of **Over-engineering** that is the act of providing a **solution** to a **problem** in an elaborate or complicated manner, where a **simpler solution** can be demonstrated to exist with the same **efficiency** and **effectiveness** as that of the **original design**

Why we may **Over-engineer** a **product/system**?



Before we start

Writing in space ...

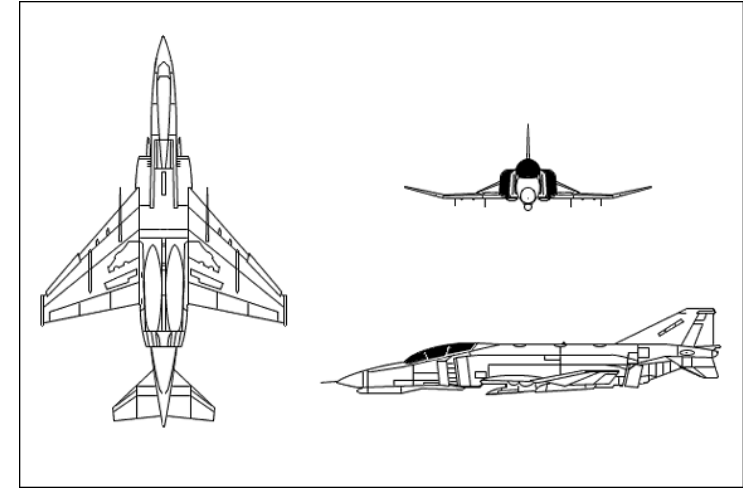
Example of **Over-engineering** that is the act of providing a **solution** to a **problem** in an elaborate or complicated manner, where a **simpler solution** can be demonstrated to exist with the same **efficiency** and **effectiveness** as that of the **original design**

Why we may **Over-engineer** a **product/system**?



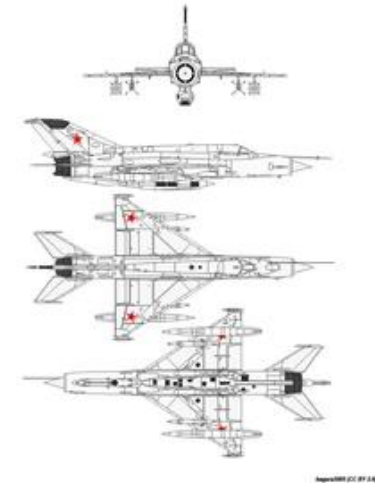
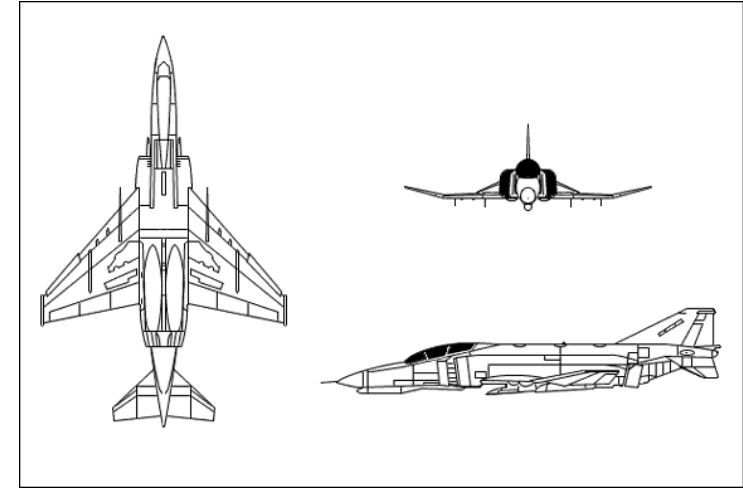
Requirements!

Before we start



Before we start

F-4 Phantom

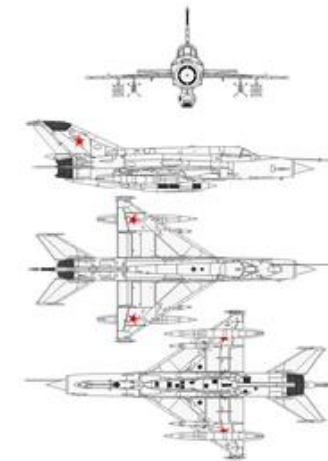
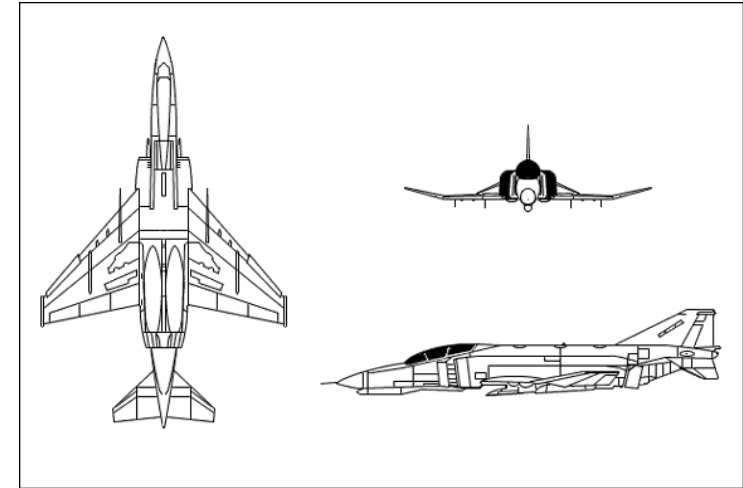


Before we start

F-4 Phantom

The **F-4 Phantom** is an American tandem two-seat, twin-engine, all-weather, long-range supersonic jet interceptor and fighter-bomber.

F4 was designed to carry air-to-air radar-guided missiles.



MIG 21 speed: 2,229 km/h

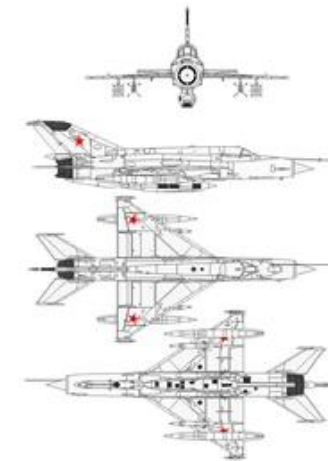
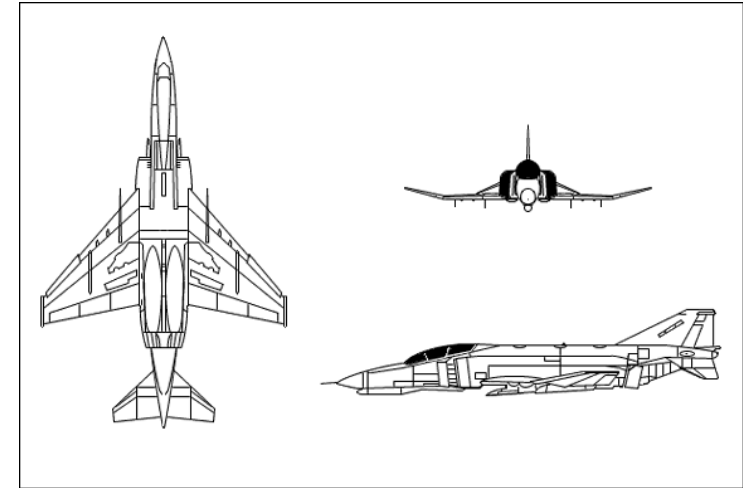
Before we start

F-4 Phantom

The **F-4 Phantom** is an American tandem two-seat, twin-engine, all-weather, long-range supersonic jet interceptor and fighter-bomber.

F4 was designed to carry air-to-air radar-guided missiles.

The relatively new **radar-guided missiles** at the time were frequently reported as **unreliable** and pilots had to **fire multiple missiles** just to hit one enemy fighter.



MIG 21 speed: 2,229 km/h

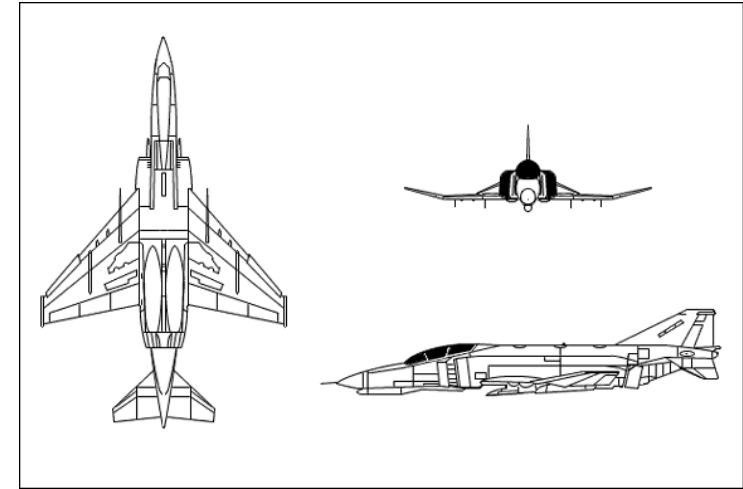
Before we start

F-4 Phantom

The **F-4 Phantom** is an American tandem two-seat, twin-engine, all-weather, long-range supersonic jet interceptor and fighter-bomber.

F4 was designed to carry air-to-air radar-guided missiles.

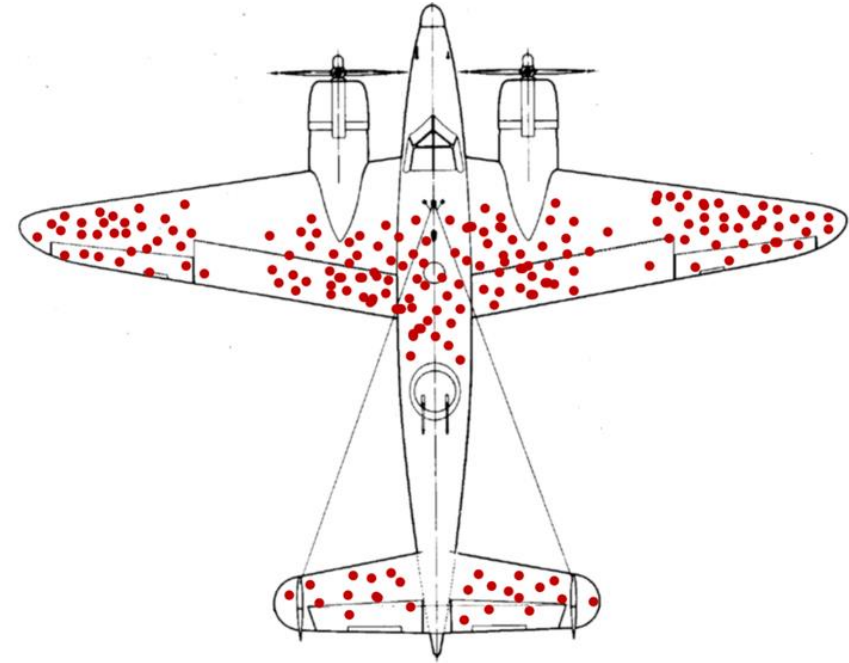
The relatively new **radar-guided missiles** at the time were frequently reported as **unreliable** and pilots had to **fire multiple missiles** just to hit one enemy fighter.



The lack of an internal gun
"was the biggest mistake" in
the F4 design

Before we start

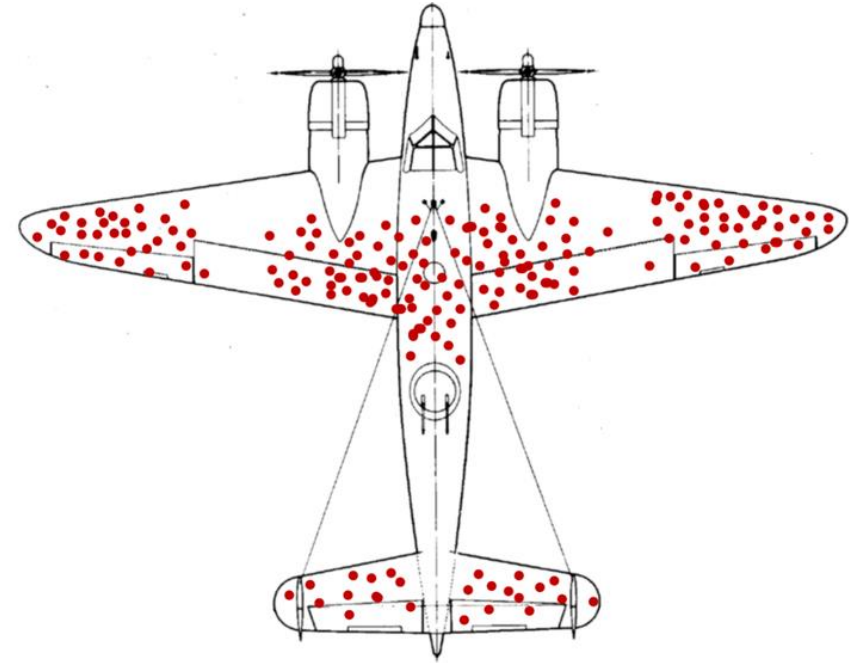
Survivorship bias



Before we start

Survivorship bias

During World War II, a research examined the damage done to aircraft that had returned from missions.

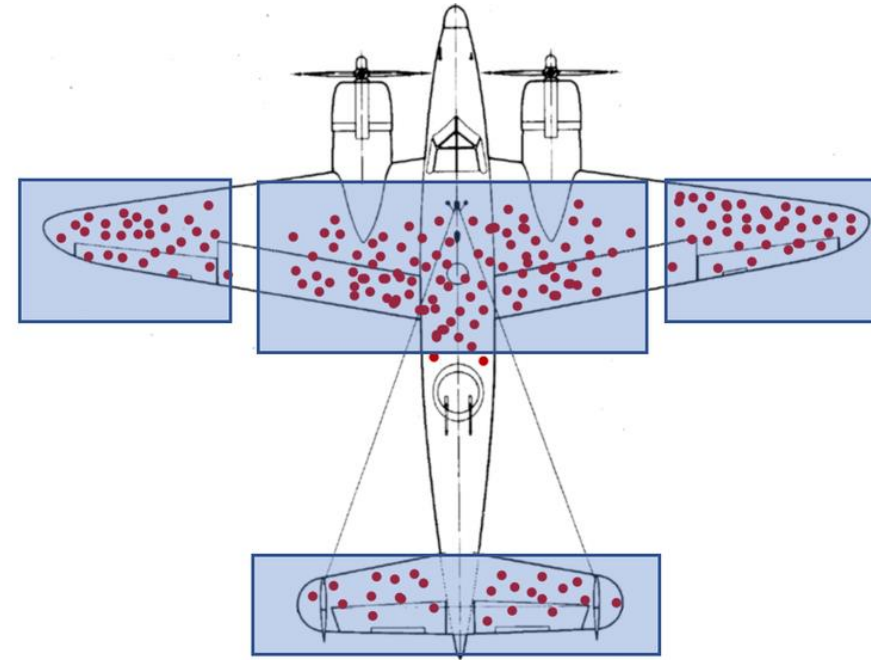


Before we start

Survivorship bias

During World War II, a research examined the damage done to aircraft that had returned from missions.

They recommended adding armor to the areas that showed the least damage.



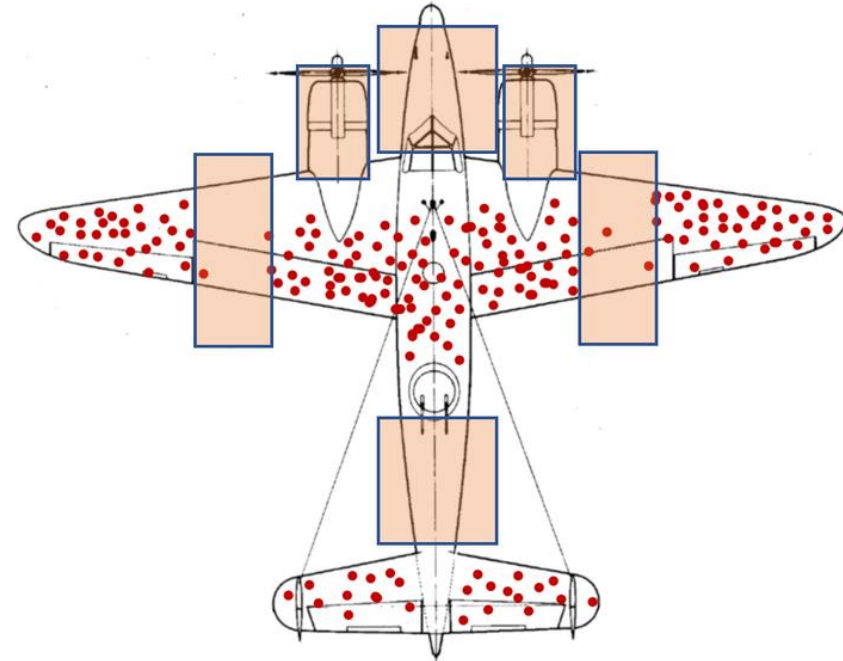
Before we start

Survivorship bias

During World War II, a research examined the damage done to aircraft that had returned from missions.

They recommended adding armor to the areas that showed the least damage.

The **fatally damaged** (did not return) planes **was being ignored**.



Survivorship bias or **survival bias** is the **logical error** of **concentrating** on entities that **passed a selection process** while overlooking **those that did not**.

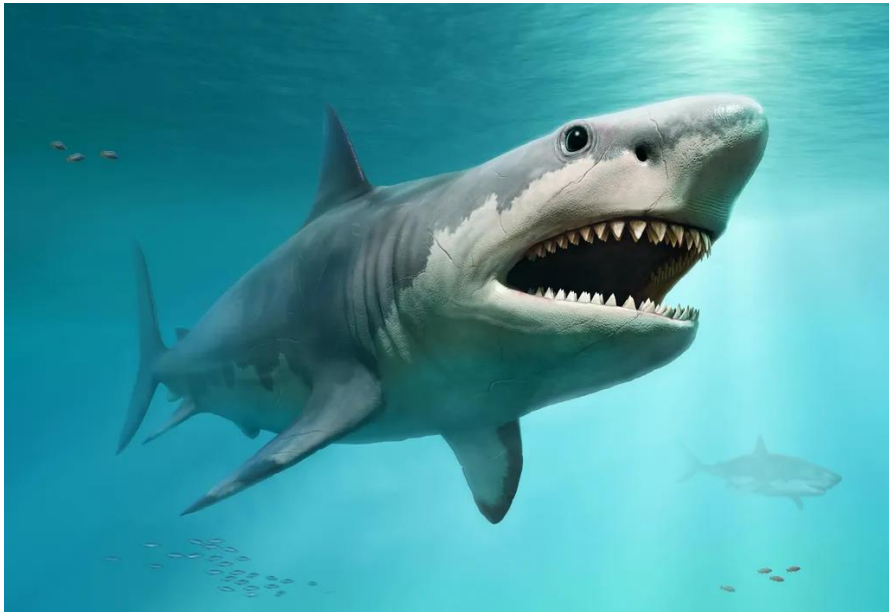


Perfect design

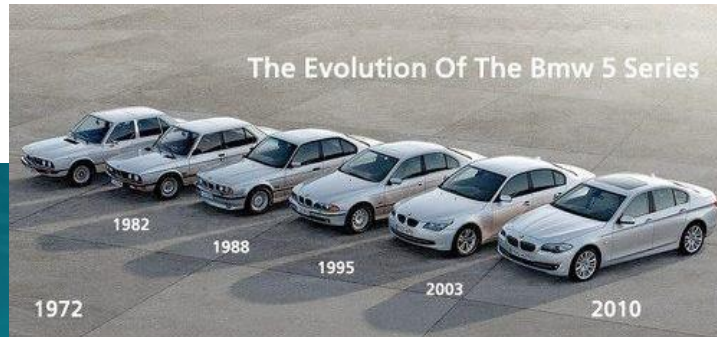
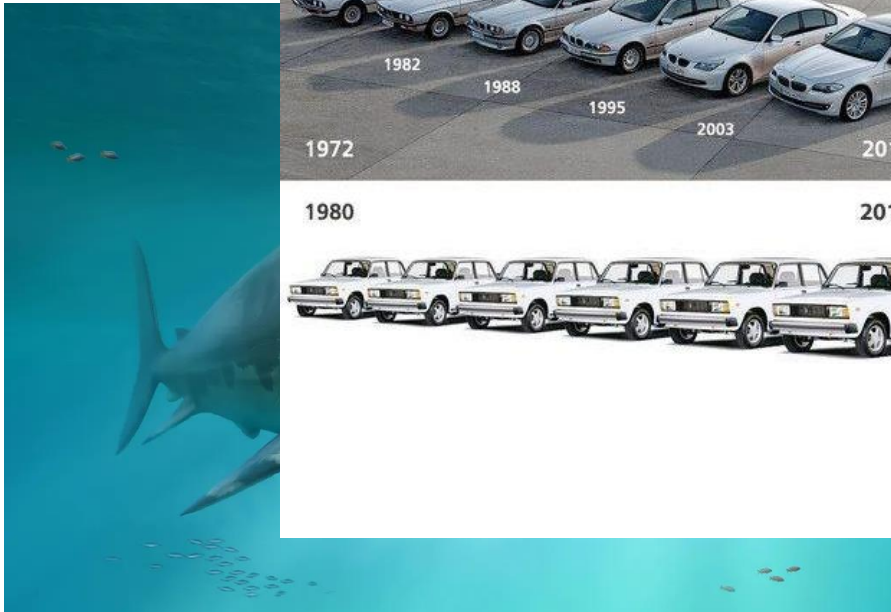
Perfect design



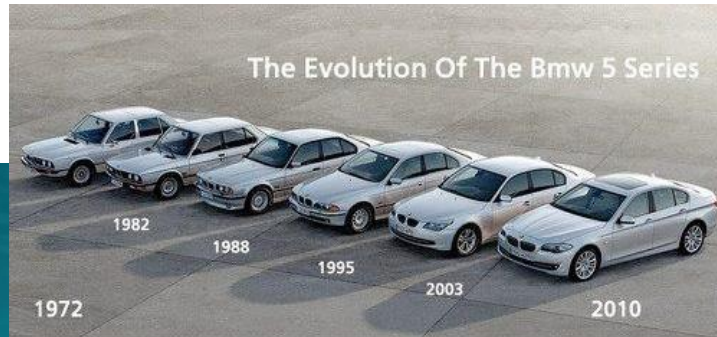
Perfect design



Perfect design



Perfect design



Mercedes S-Class by the generation



LADA did not need EVOLVE ... because it is perfect from the beginning



Outline

Part 1 – System Development Life Cycle (SDLC).

Part 2 – [Enterprise] System requirements.

Part 3 – [Enterprise] System Architecture.

Part 4 – Models and modeling

Part 5 – Domain models and domain modeling.

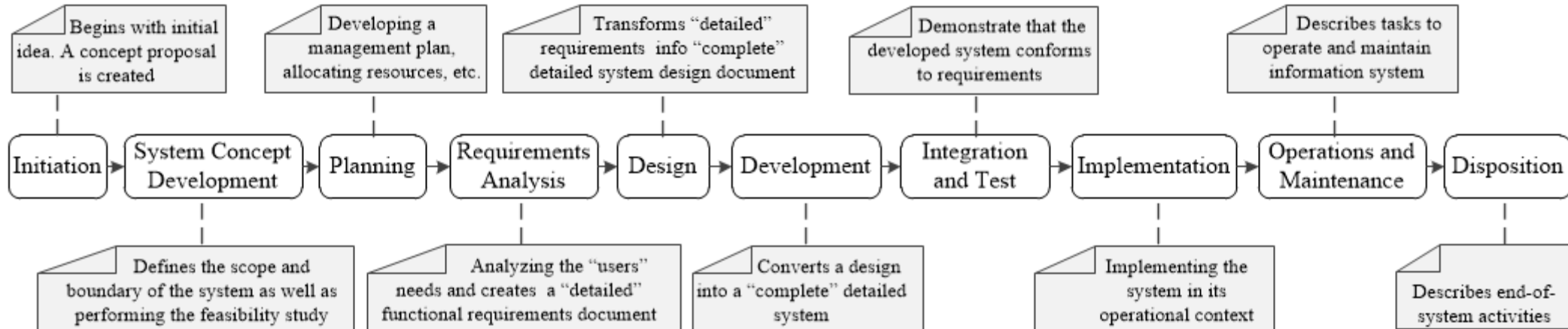
Part 1

System Development Life Cycle

System Development Life Cycle (SDLC)



System Development Life Cycle (SDLC)



System Development Life Cycle (SDLC): includes ten phases through which the products are created/modified.

- 1. Initiation** Begins with initial idea, and a concept proposal is created.
- 2. System Concept Development:** defines the scope and boundary of the system as well as performing the feasibility study.
- 3. Planning:** developing a management plan, allocating resources, etc.
- 4. Requirements Analysis:** analyzing the "users" needs and creates a "detailed" functional requirements document.
- 5. Design:** transforms "detailed" requirements info "complete" detailed system design document.
- 6. Development:** converts a design into a "complete" detailed system.
- 7. Integration and Test:** demonstrate that the developed system conforms to requirements.
- 8. Implementation:** implementing the system in its operational context.
- 9. Operations and Maintenance:** describes tasks to operate and maintain information system.
- 10. Disposition:** describes end-of-system activities.

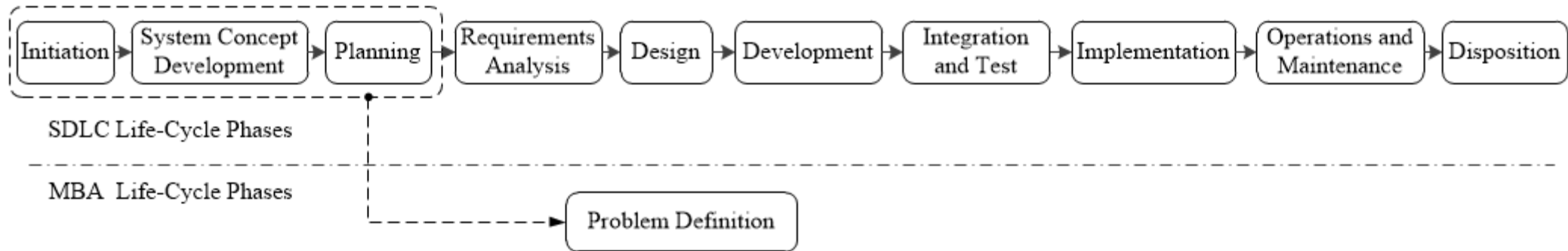
SDLC/MBA Life-Cycle phases



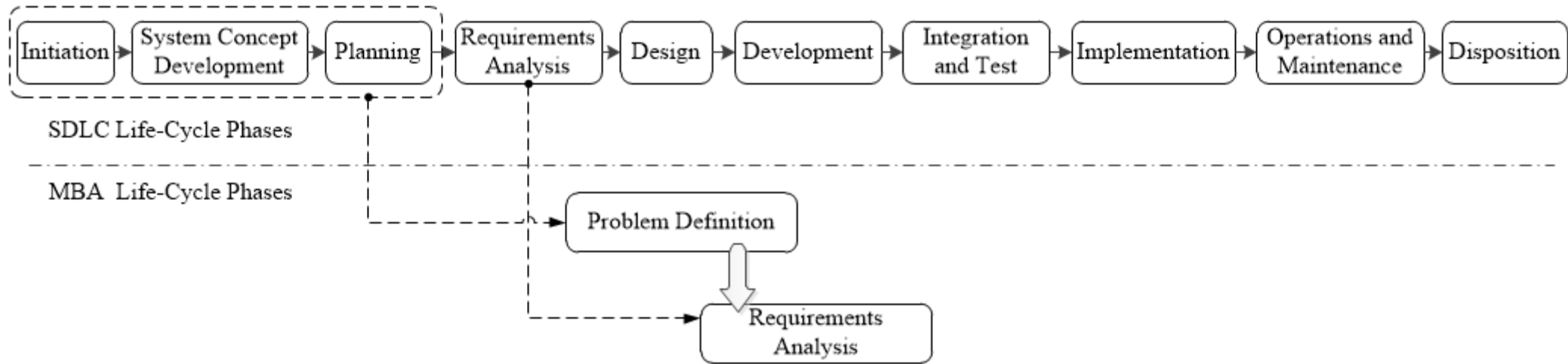
SDLC Life-Cycle Phases

MBA Life-Cycle Phases

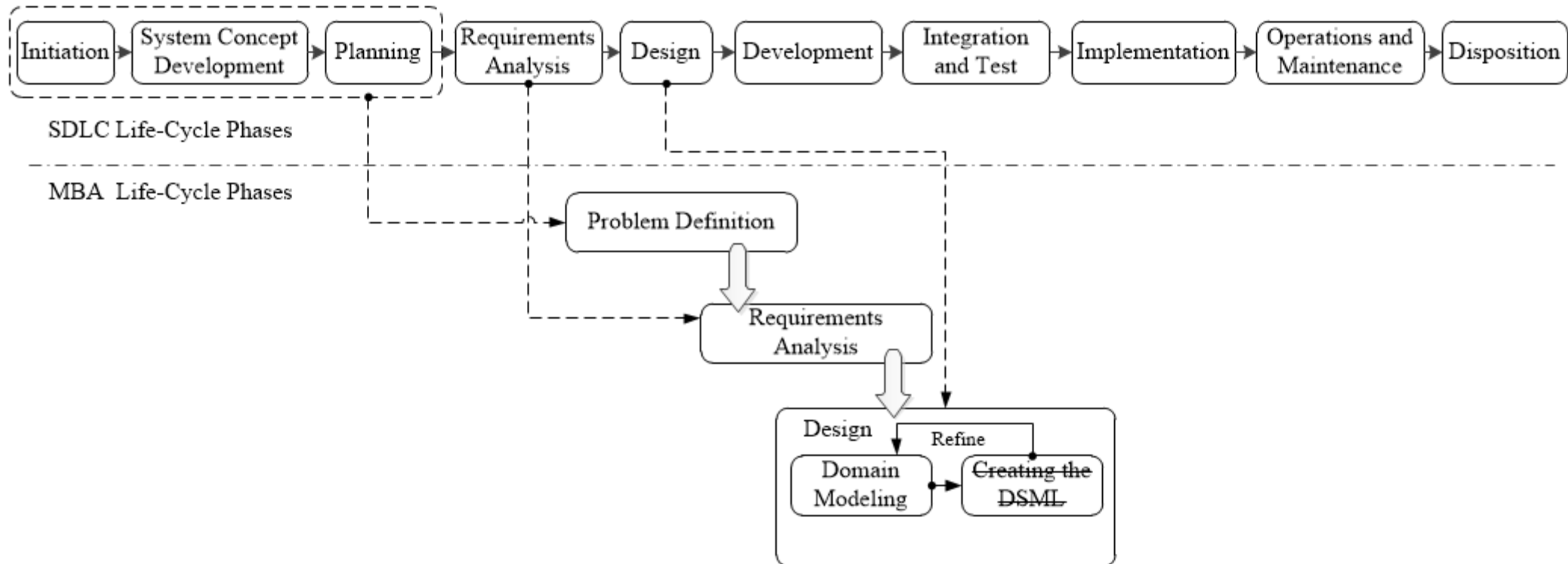
SDLC/MBA Life-Cycle phases



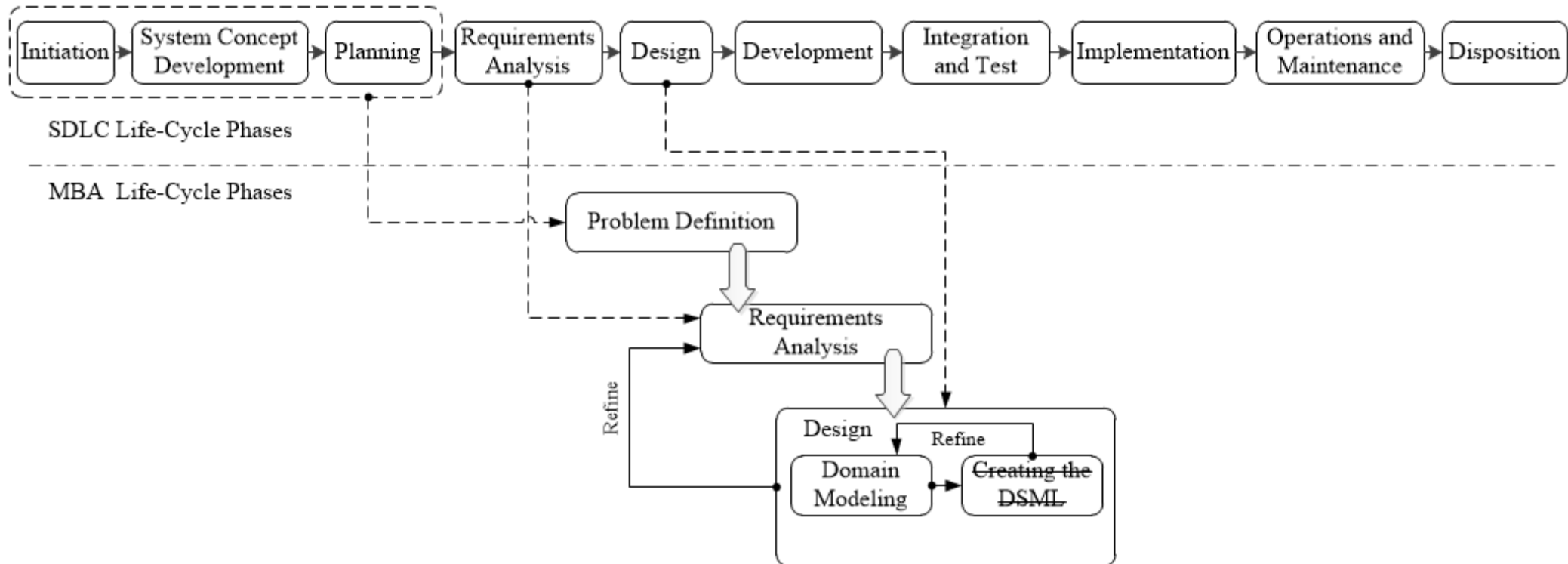
SDLC/MBA Life-Cycle phases



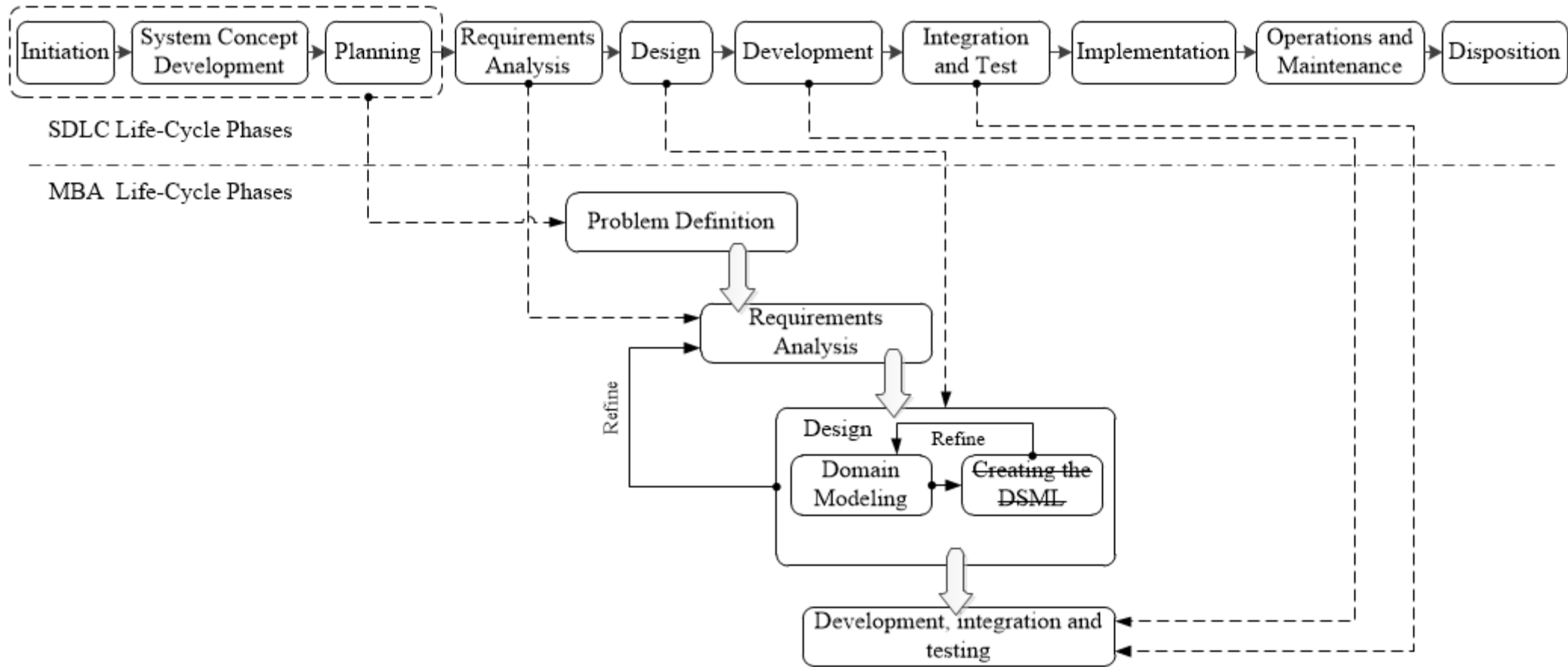
SDLC/MBA Life-Cycle phases



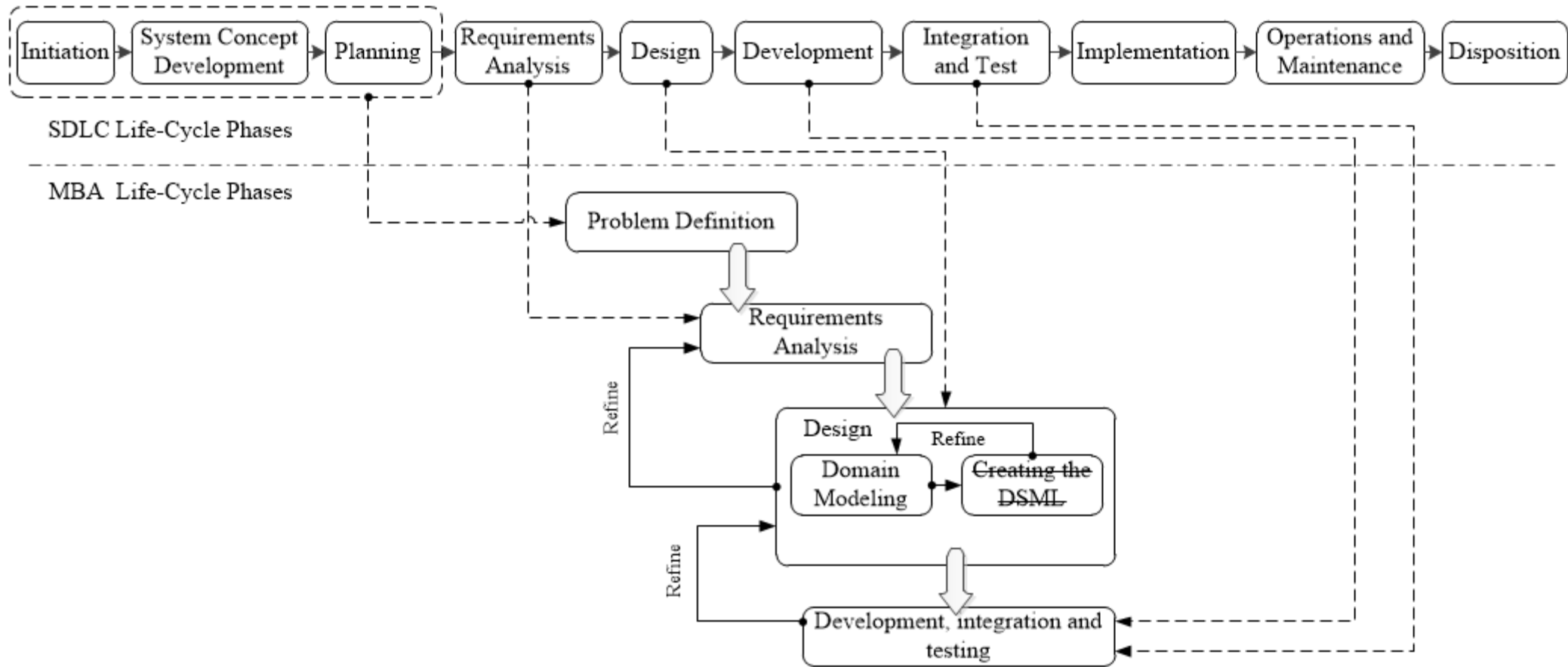
SDLC/MBA Life-Cycle phases



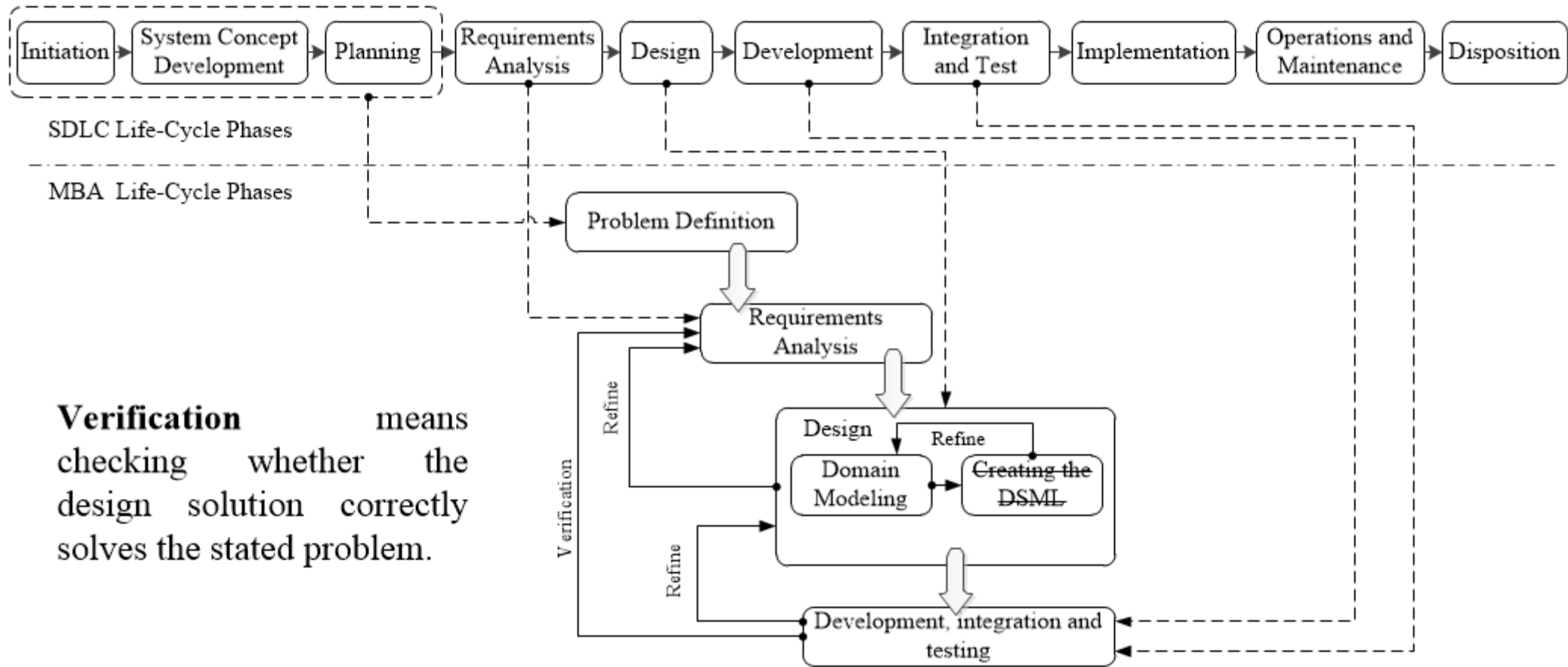
SDLC/MBA Life-Cycle phases



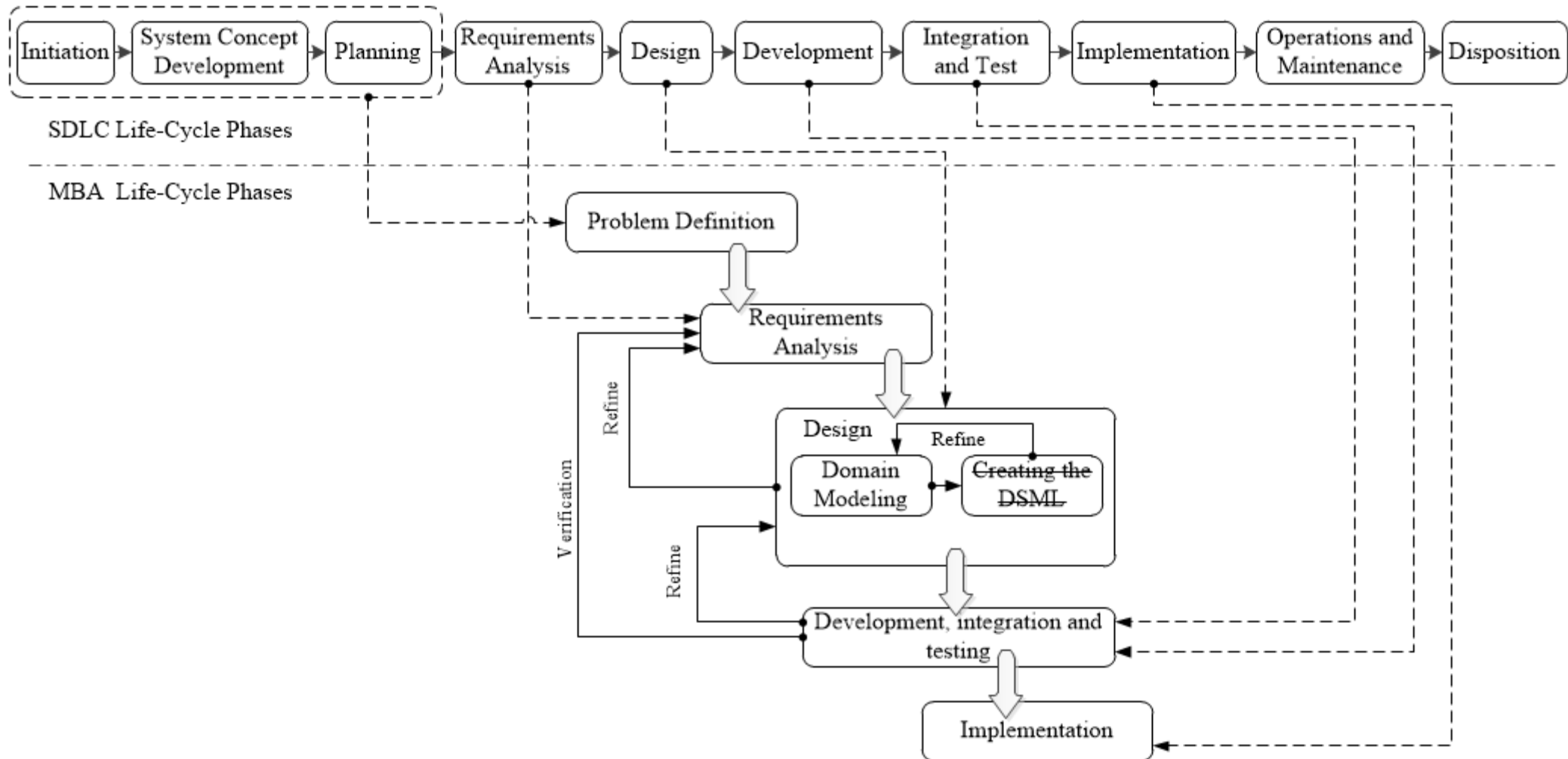
SDLC/MBA Life-Cycle phases



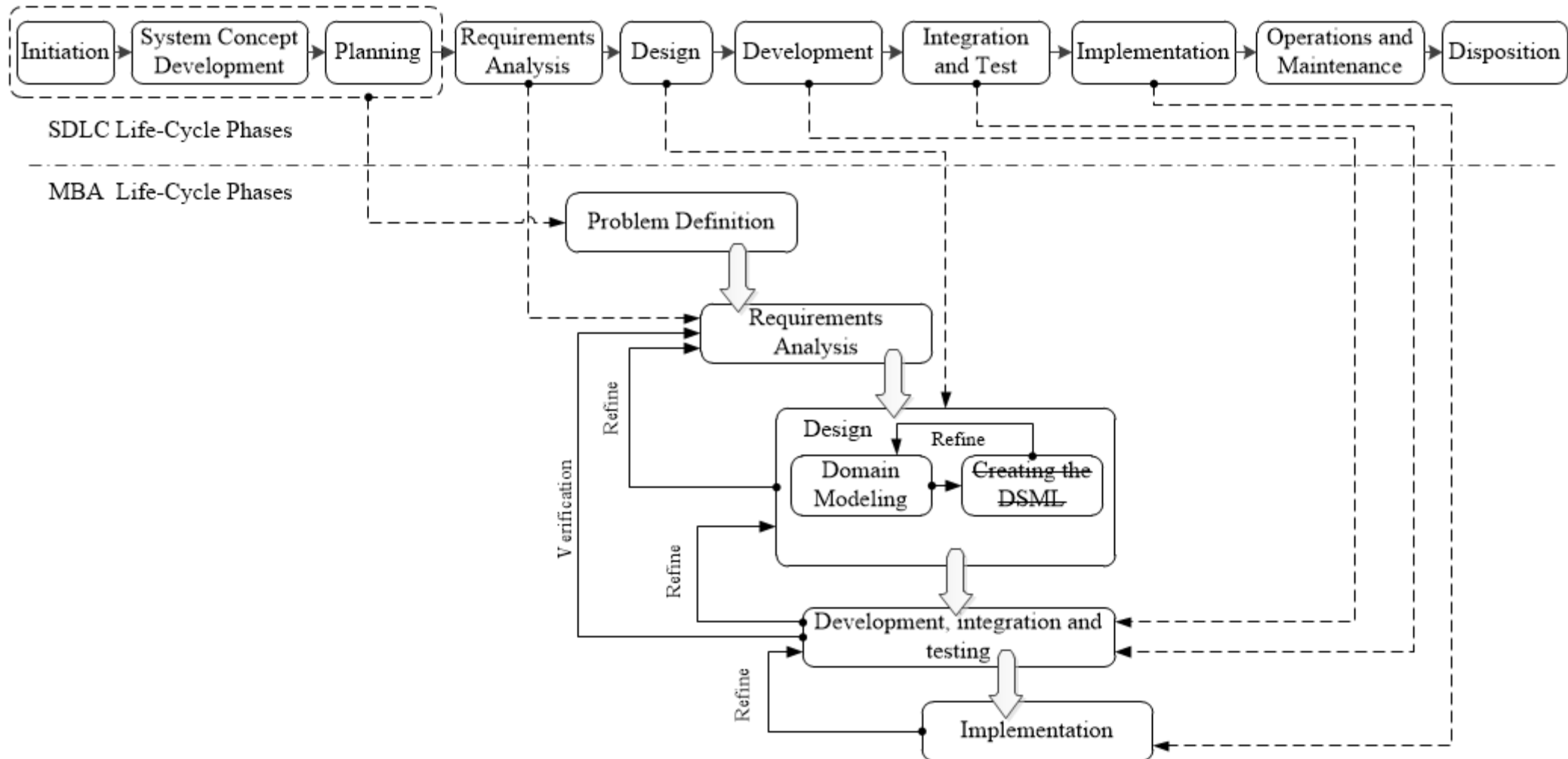
SDLC/MBA Life-Cycle phases



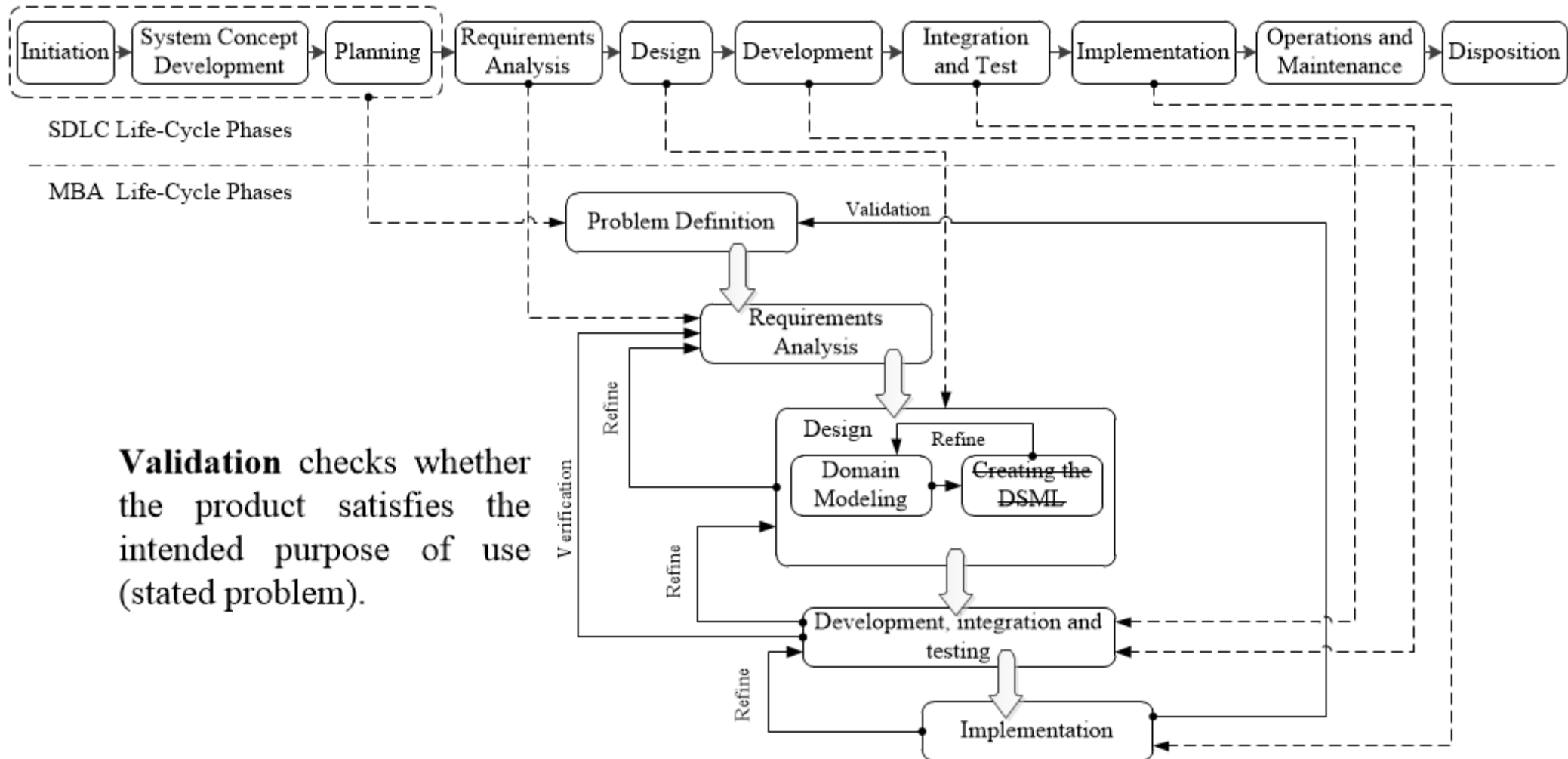
SDLC/MBA Life-Cycle phases



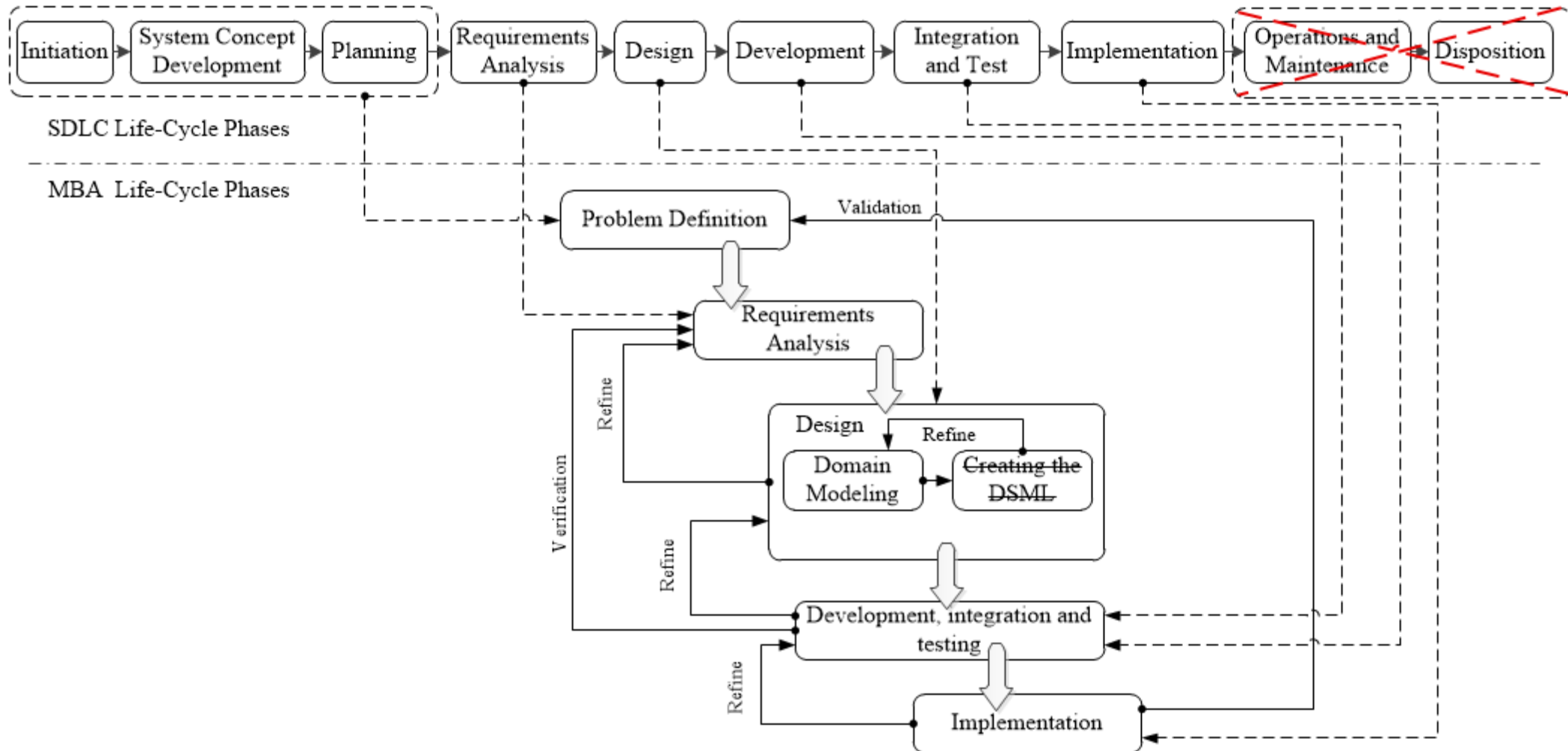
SDLC/MBA Life-Cycle phases



SDLC/MBA Life-Cycle phases

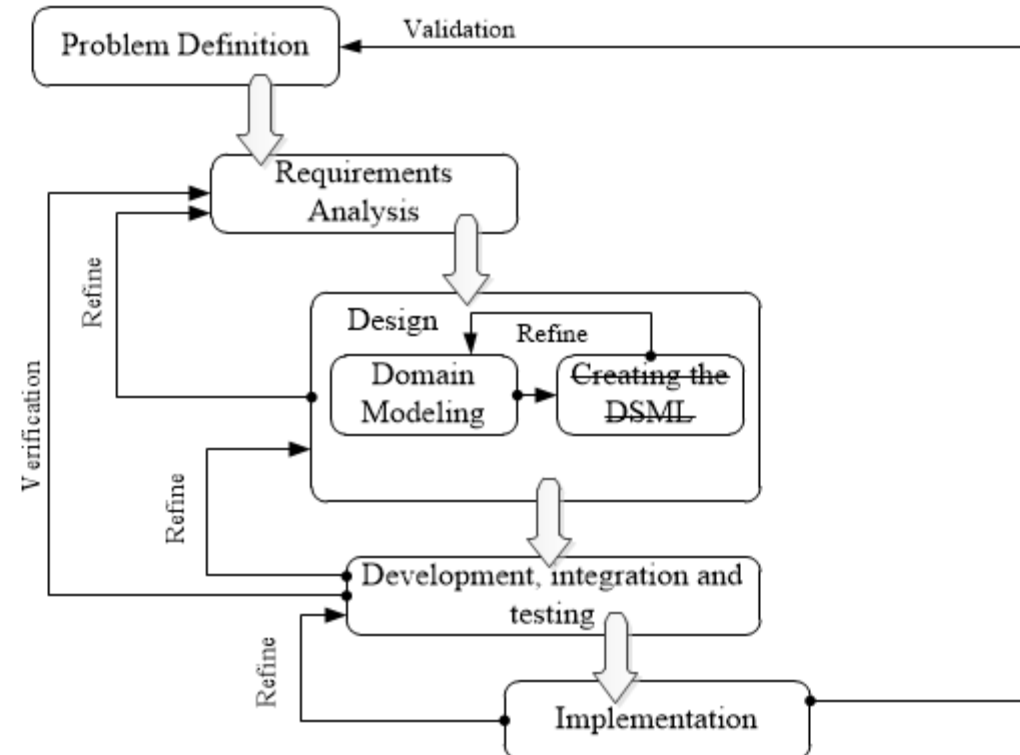


SDLC/MBA Life-Cycle phases



MBA Life-Cycle phases

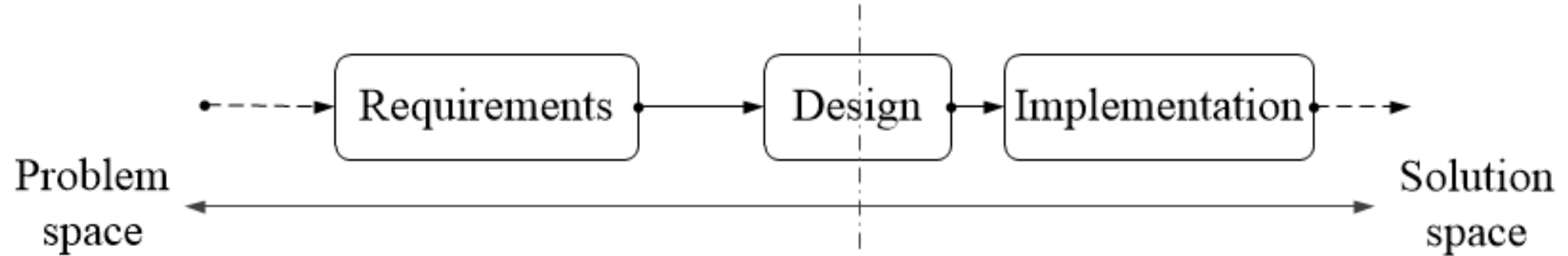
MBA Life-Cycle Phases



Part 2

1. [Enterprise] System requirements

What Are Requirements?



Requirements constitute a **specification** for the new **system***

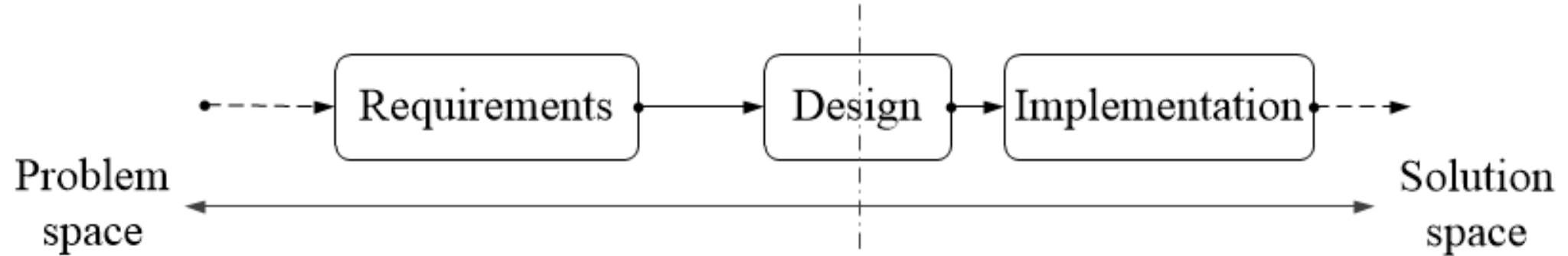
A **problem** can be defined as the difference between things as *they are now* and things as *they are desired*.

*It is more important to understand the **problem** than the **solution***

- Albert Einstein

*Ross, D. T., & Schoman, K. E. (1977). Structured analysis for requirements definition. *IEEE transactions on Software Engineering*, (1), 6-15.

What Are Requirements?



Requirements ...must identify:

why a system is needed, based on current and foreseen conditions;

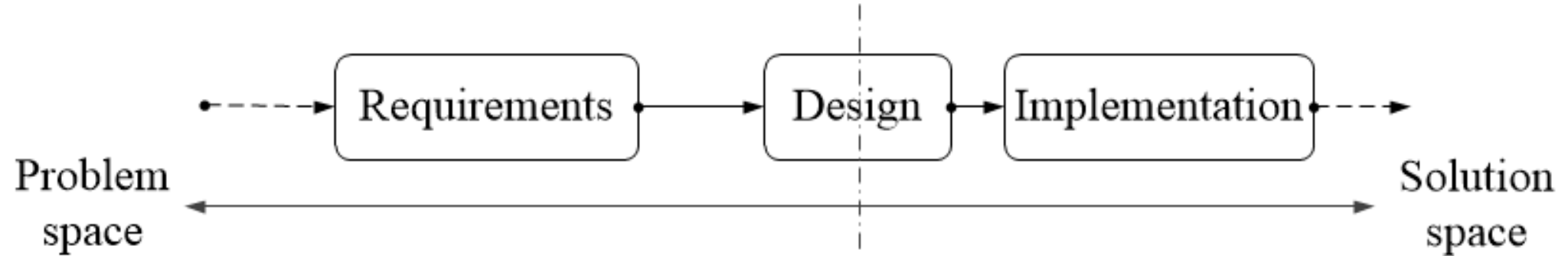
what system features will serve and satisfy this context;

how the system is to be constructed.

*A problem **unstated** is a problem **unsolved***

- Douglas T. Ross

What Are Requirements?



According to Jackson*, the term **requirements** might refer to many things:

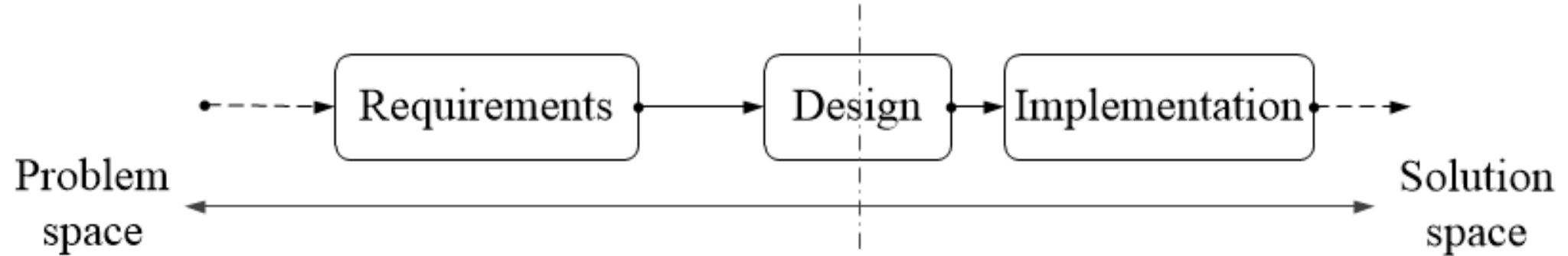
- The system must tolerate user errors.
- An order request should not be completed unless the cost has been paid.

Requirement should be **satisfiable**.

- An order request should not be completed unless the cost has been paid.
- Can be **satisfied** by implimenting a mechanisim to check the payment method, withdraw the cost befor completing the purchase order.

*M. Jackson, "The Meaning of Requirements," Annals of Software Engineering, Vol 3, pp5-21, Baltzer Science Publishers. 1997.

What Are Requirements?



There are **two** main types of requirements:

- **Functional** requirements: describe **what** the system is supposed to do;
- **Non-functional** requirements: describe **how** the system will do it.

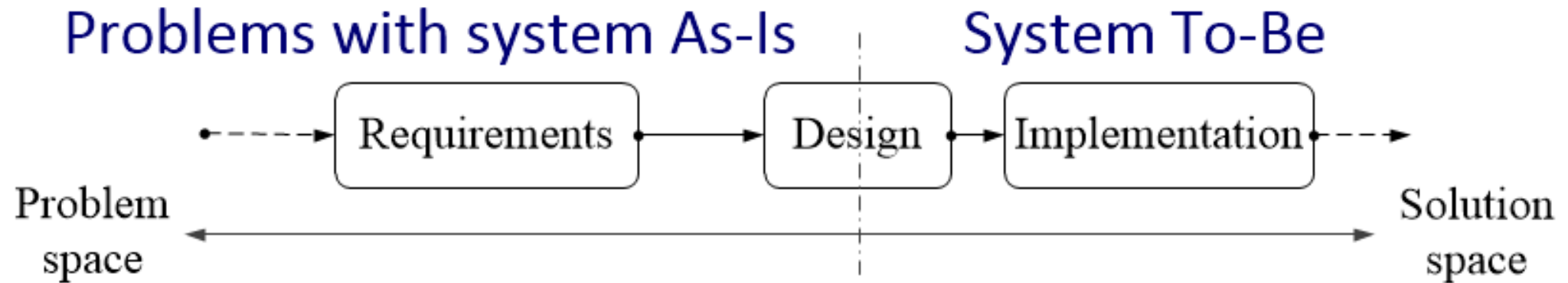
Functional requirements:

Describe the functionality, inputs/outputs, etc. of the new system.

Non-functional requirements:

Describe safety/reliability/security needs, performance/usability criteria, etc.

What is Requirements Engineering?



Requirements Engineering (RE) refers to the process of “defining”, documenting and “maintaining” requirements*.

Requirements Engineering is about determining[#]

- problems with the current status (As-Is);
- objectives to achieve;
- changes to bring about for a better future (To-Be).

* B. A. Nuseibeh and S. M. Easterbrook, "Requirements Engineering: A Roadmap", In A. C. W. Finkelstein (ed.) *The Future of Software Engineering*. (Companion volume to the proc. of ICSE'00). IEEE Computer Society Press.

[#] Easterbrook, Steve. "What are requirements. "Draft Chapter, <http://www.cs.toronto.edu/~sme/papers/2004/FoRE-chapter02-v7.pdf> (2004).

Why Requirements Engineering?

Bad requirements is a main reason for a project failures. **Errors in requirements propagate** to other activities.

cost (correcting implementation errors) = **100 X cost** (correcting req. errors)*.

Errors should be **detected** and **addressed** at requirements level rather than after implementing the system.

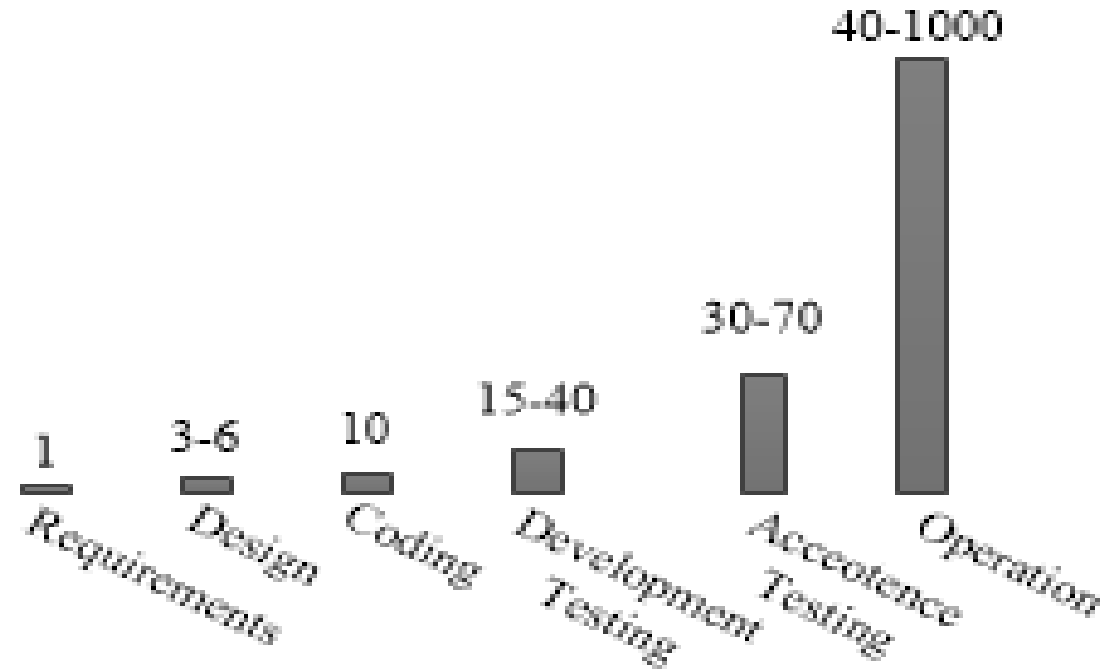
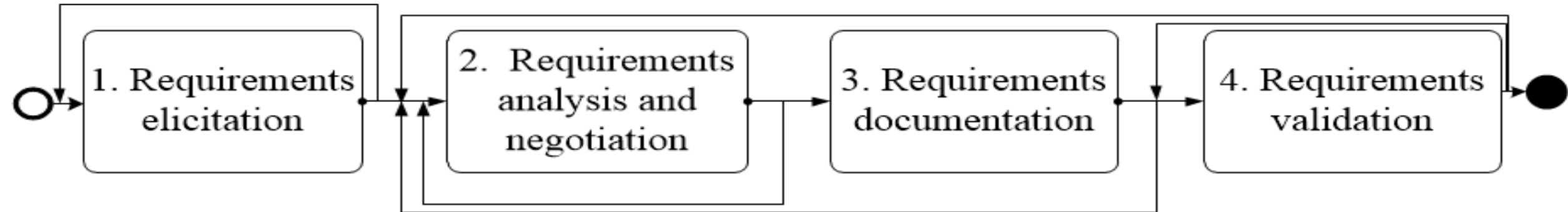


Fig. Relative cost to fix software errors per life cycle phase [5]

* Boehm, B. W. (1981). *Software engineering economics* (Vol. 197). Englewood Cliffs (NJ): Prentice-hall.

How to do Requirements Engineering?



Requirements Engineering Process*:

- 1. Requirements elicitation:** requirements discovered through consultation with stakeholders;
- 2. Requirements analysis and negotiation:** requirements are analyzed and conflicts resolved through negotiation;
- 3. Requirements documentation:** a requirements document is produced;
- 4. Requirements validation:** The requirements document is checked for consistency and completeness.

* Kotonya, G. et al. *Requirements Engineering: Processes and Techniques*, John Wiley & Sons, 1998.



[Enterprise] System requirements –PE

R1. The system should show products, their prices, etc. in a product catalogue.

R2.

[Enterprise] System requirements –PE

- R1.** The system should show products, their prices, etc. in a product catalogue.
- R2.** The system should allow a customer to check the availability of a given product.
- R3.** The system should allow a customer to check whether the product can be delivered to a specific location.
- R4.** The system should allow a customer to purchase a product by completing a purchase order.
- R5.** The system should allow a customer to modify/cancel its order within 24 hours.
- R6.** The system should allow a customer to request modifying/cancelling its order within 24-48 hours. Such request can be approve/reject by an employee.
- R7.** The system should issue an invoice for each purchase order.
- R8.** The system should allow a customer to pay for her purchase invoice by various means.
- R9.** A successful purchase order results in a delivery order.
- R10.** An inventory employee can change the status of a delivery order.

Product purchase scenario

- A customer can visit the product catalogue that list the available products, their prices and delivery possibilities.
- A customer can select a product to purchase, which results in a purchase order.
- A purchase order can be successful if the product is available, can be delivered to the customer, and the customer pays for its corresponding invoice.
- A customer can automatically modify/cancel its order within 24 hours from the purchase.
- A customer can request modifying/cancelling its order within 24-48 hours. Such request need to be approved by an employee.
- A successful purchase order results in a delivery order.

This scenario and derived requirements will help you to understand how you can start analysing your system to develop its corresponding domain model.

Part 3

3.1 [Enterprise] System Architecture

[Enterprise] System Architecture

- An architecture is "the fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution^{*,#}."
- **[Enterprise] System architecture** is an abstract representation of the system components, their relationships, and the principles governing its design and evolution.

*Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure — **Melvin E. Conway***

*ANSI/IEEE 1471-2000, "Recommended Practice for Architecture Description of Software-Intensive Systems."

#ISO/IEC 42010:2007, "Systems and Software Engineering—Recommended Practice for Architectural Description of Software-Intensive Systems."

[Reading] Kruchten, Philippe B. "The 4+ 1 view model of architecture." *IEEE software* 12.6 (1995): 42-50.

[Enterprise] System Architecture



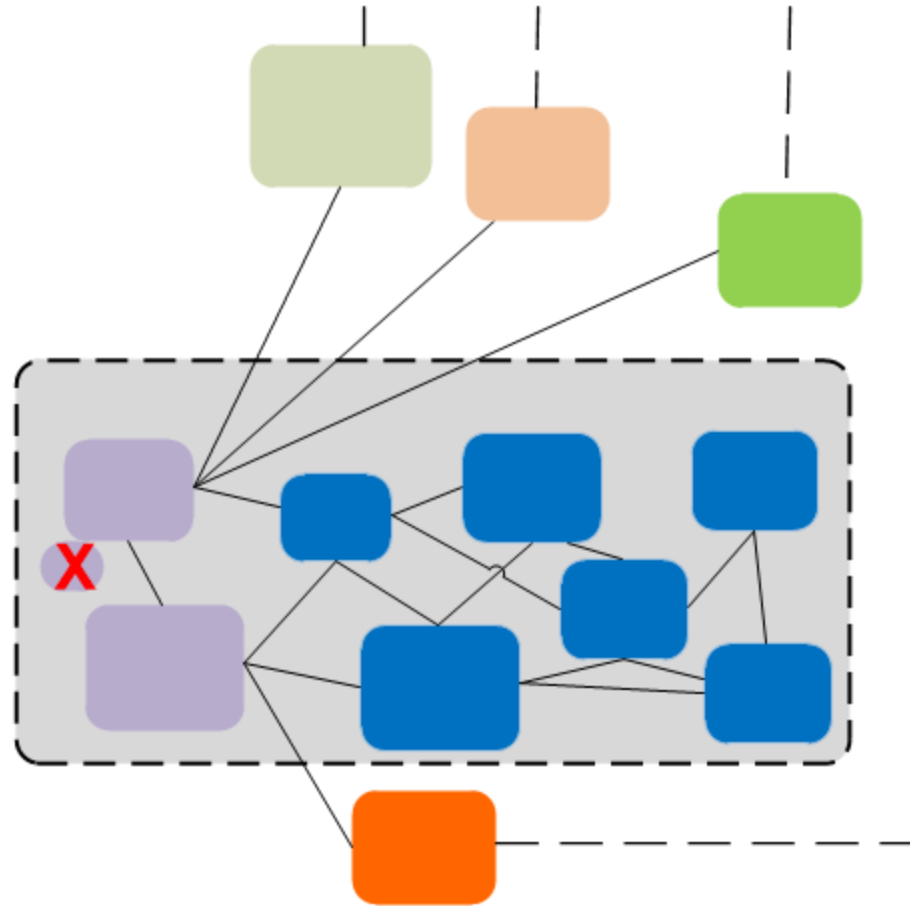
Do we need an [enterprise] system architecture?

How to define Enterprise Architecture

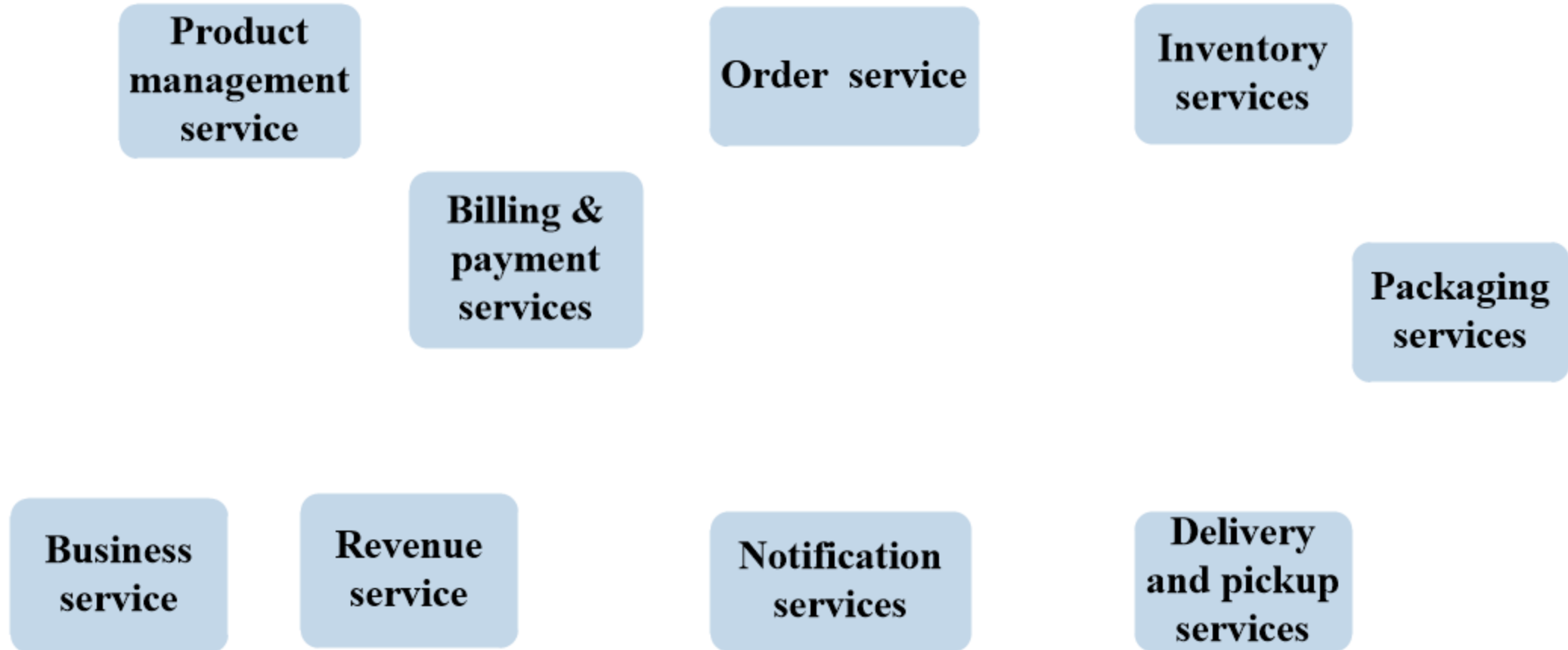
The process of constructing an enterprise architecture involves,

- identify key **components (building blocks)** that compose the enterprise system;
- identify the **relationships** among these components;
- identify how these components use each other's functionality and data to achieve their own functionalities;

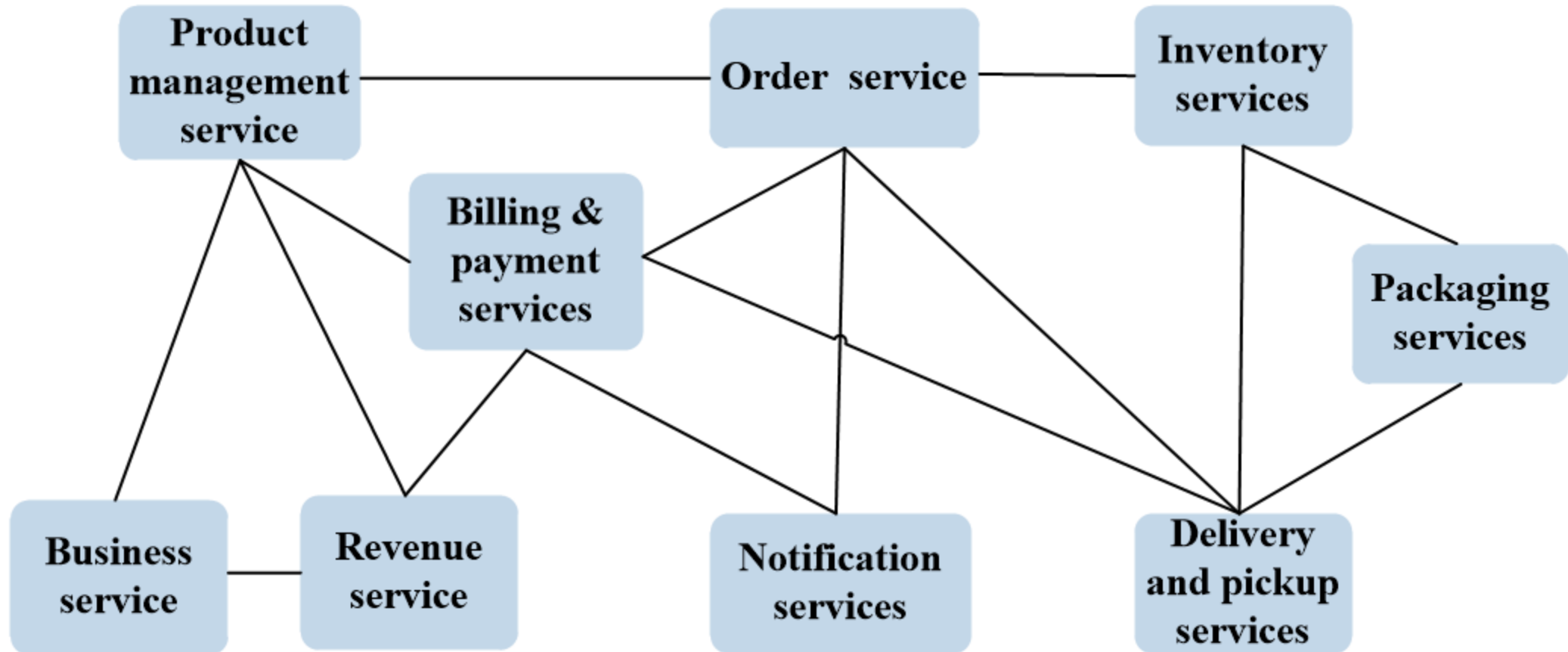
[Enterprise] System Architecture - PE



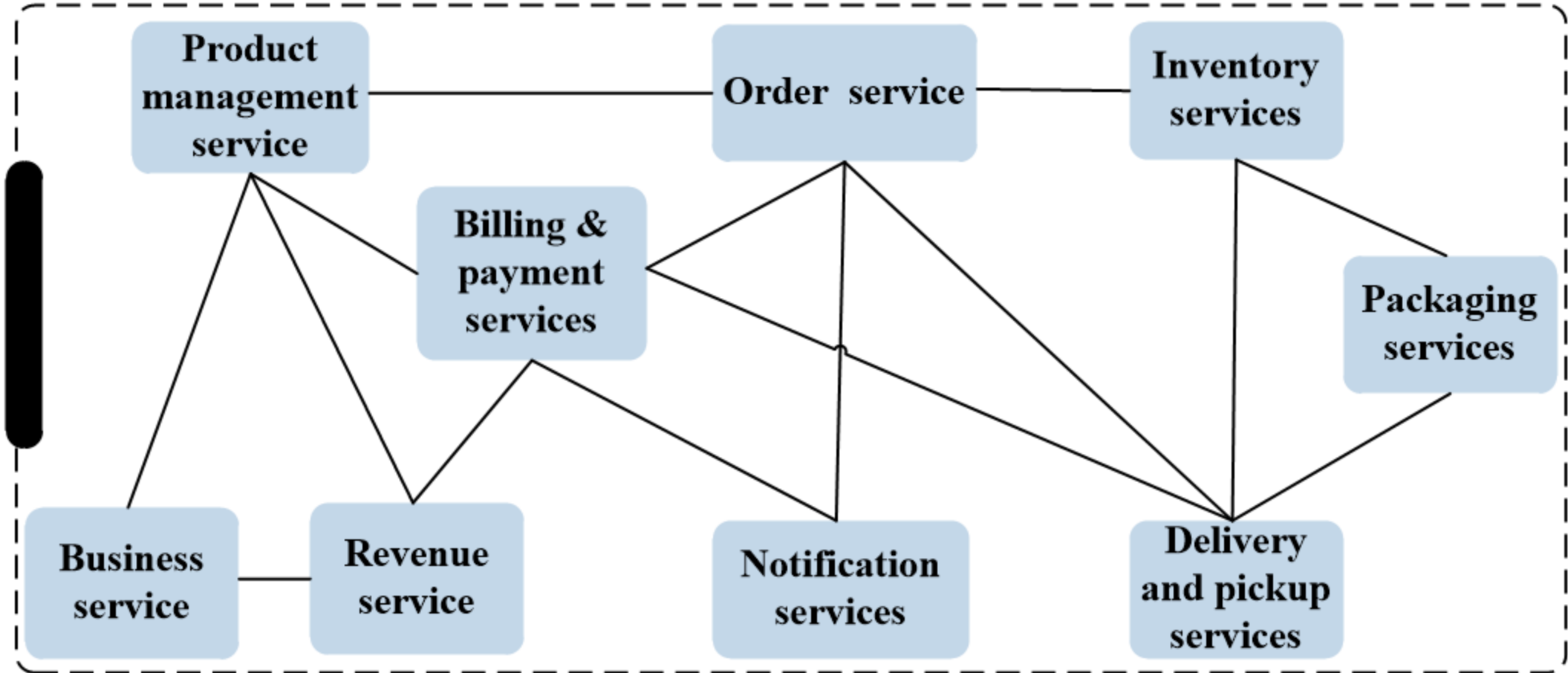
[Enterprise] System Architecture - PE



[Enterprise] System Architecture - PE



[Enterprise] System Architecture - PE





Part 4

Models and modeling



Models and modeling

Models:

Modeling:

Models and modeling

Models:

- A **model** is an **abstraction** of **reality**. Usually, **models** do not represent all aspects of **reality**.
- This allows for dealing with the real world in a simplified manner, avoiding the **complexity** and **danger** of reality.

Modeling:

- **Modeling** is a cost-effective use of **something** in place of **something** else for some purpose.
- **Modeling** allows for the use of something that is **simpler**, **safer**, and/or **cheaper** than **reality** instead of **reality** for some purpose.

Models and modeling

Modeling is supposed to represent a particular reference (“domain”) for a particular purpose in a cost-effective (feasible) way*.

Open questions:

How to represent the reference (“domain”)?

WHY we are modeling?

How to define the purpose of the modeling?

WHAT are we modeling?

How to guarantee it is cost-effective?

HOW the model can be of **high-quality** and still **feasible** to be used?

*Rothenberg J, Widman LE, Loparo KA, Nielsen NR. *The nature of modeling*. in Artificial Intelligence, Simulation and Modeling. 1989. [[Simple and very interesting paper about modeling](#)].

Models and modeling



Domain
("real world")

Models and modeling



Domain
(“real world”)

Models and modeling

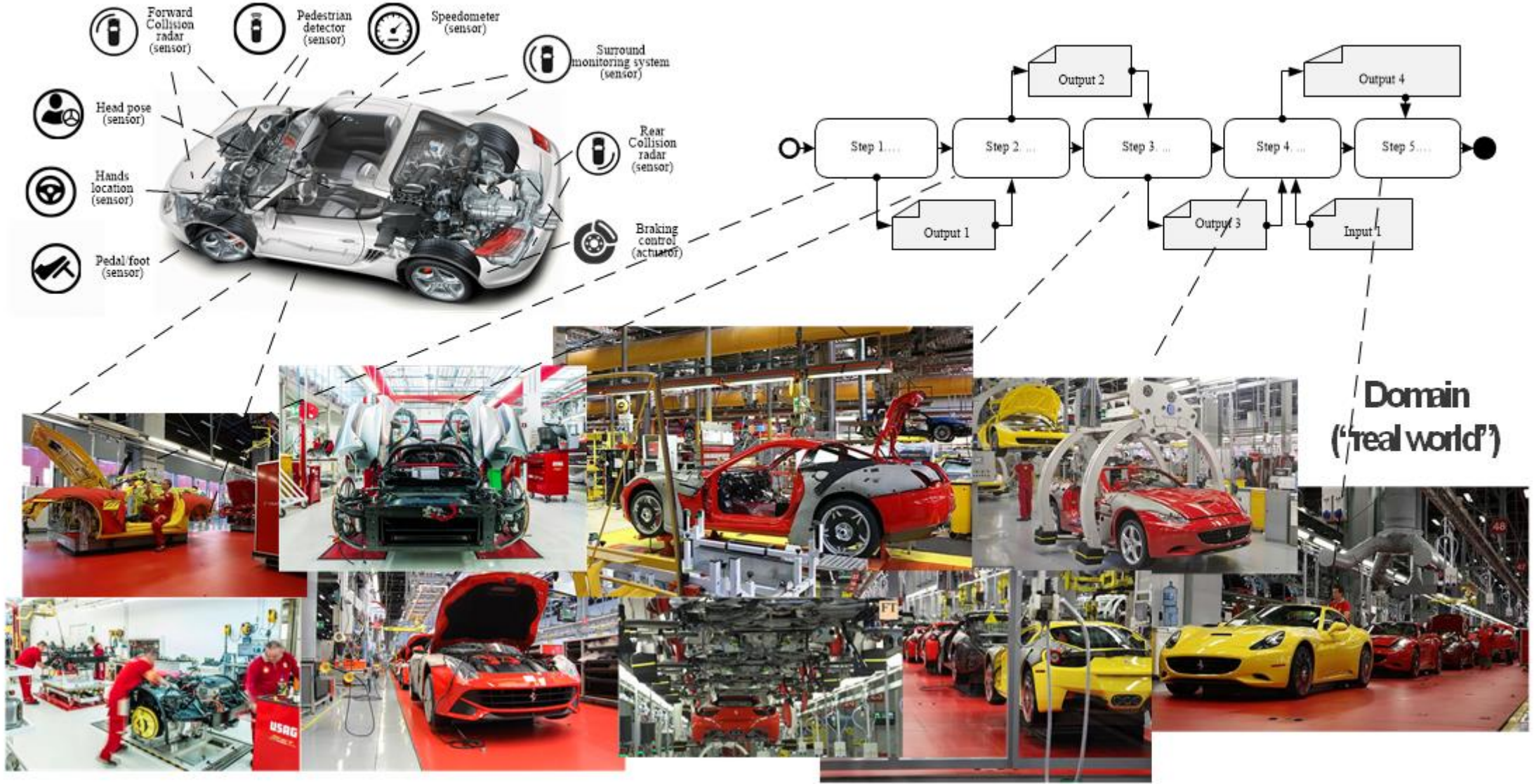


Domain
("real world")

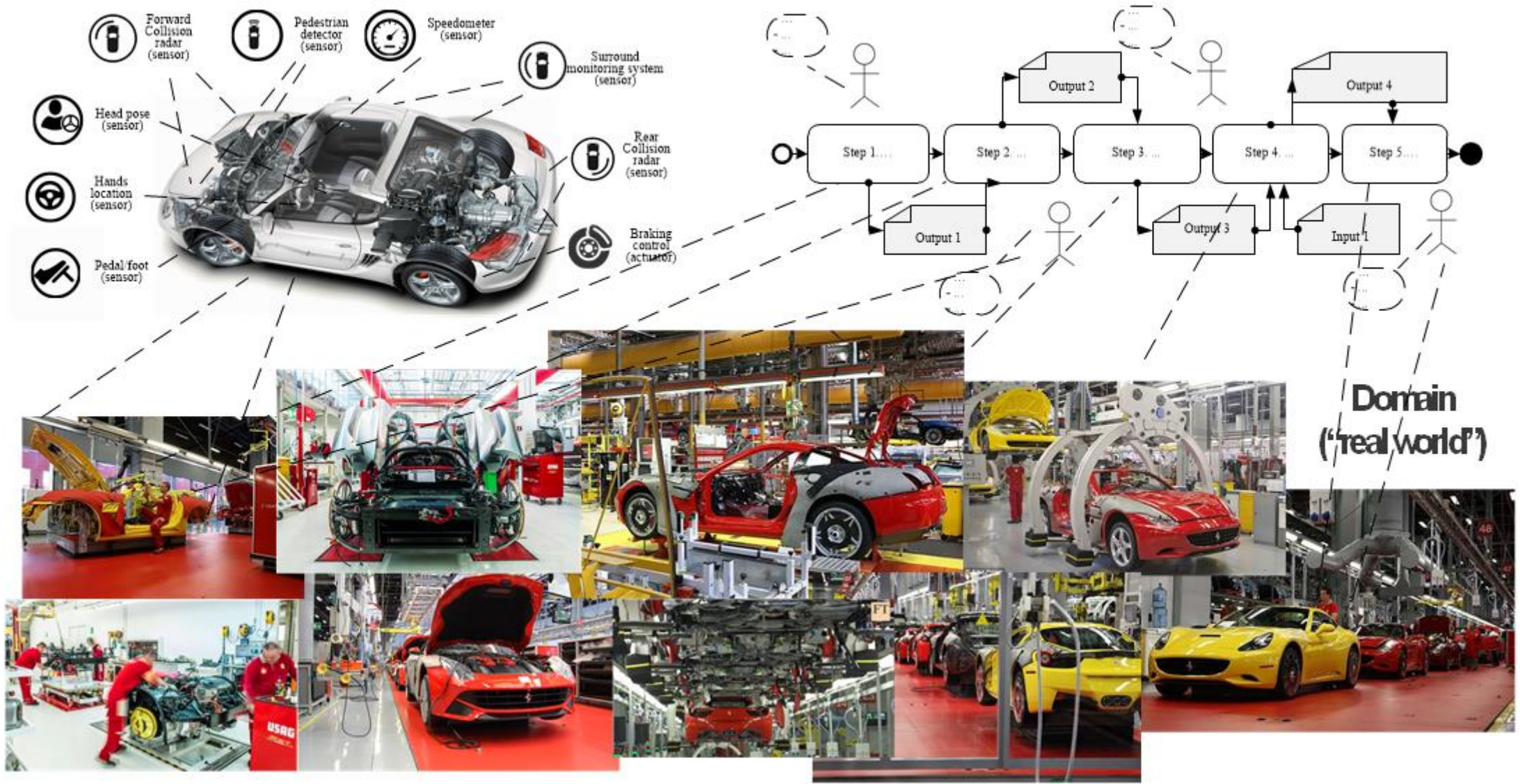




Models and modeling

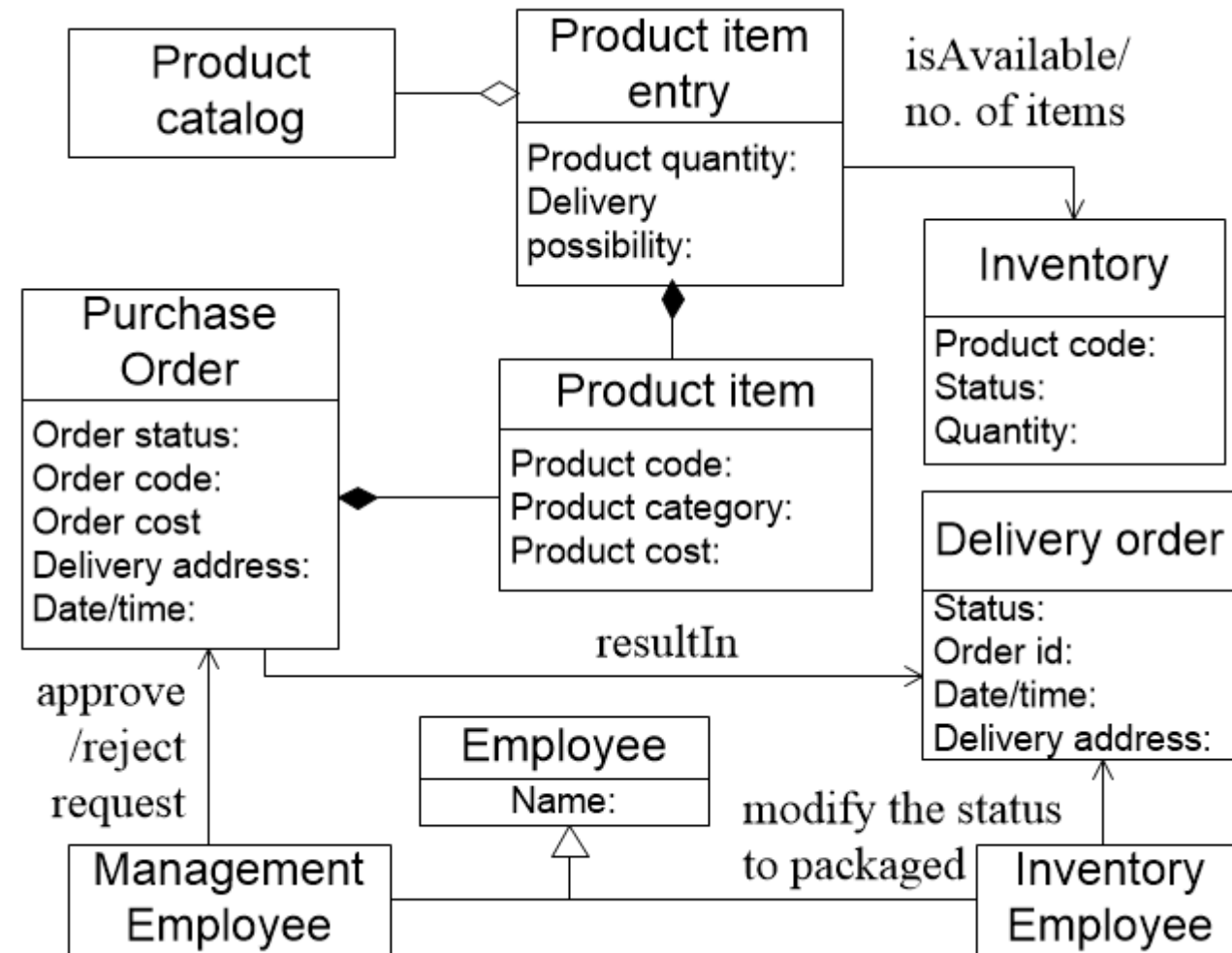


Models and modeling



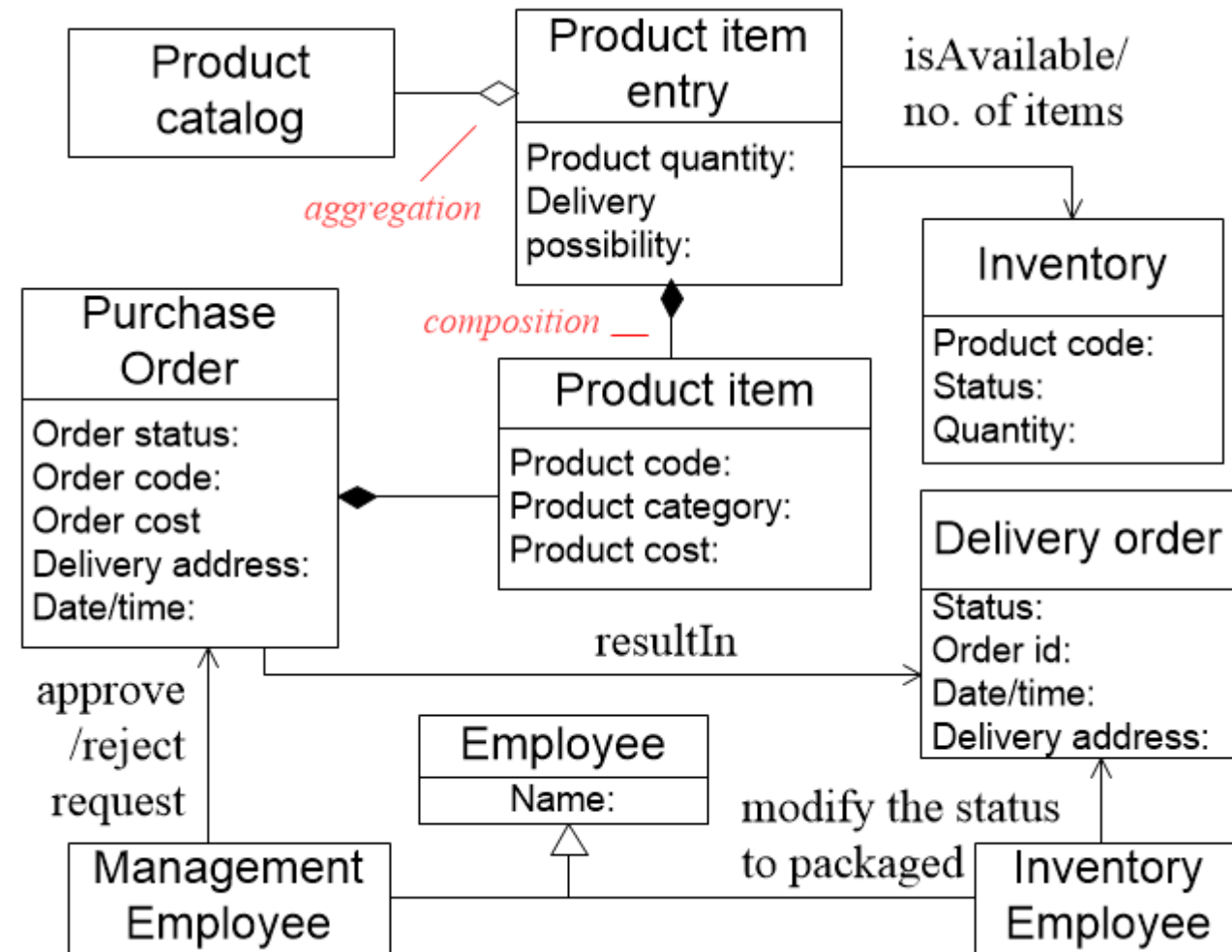
Class diagrams – UML

A class diagrams can be used to describe a system in terms of the entities “composing” it and their inter-relationships.



Class diagrams – UML

A class diagrams can be used to describe a system in terms of the entities “composing” it and their inter-relationships.

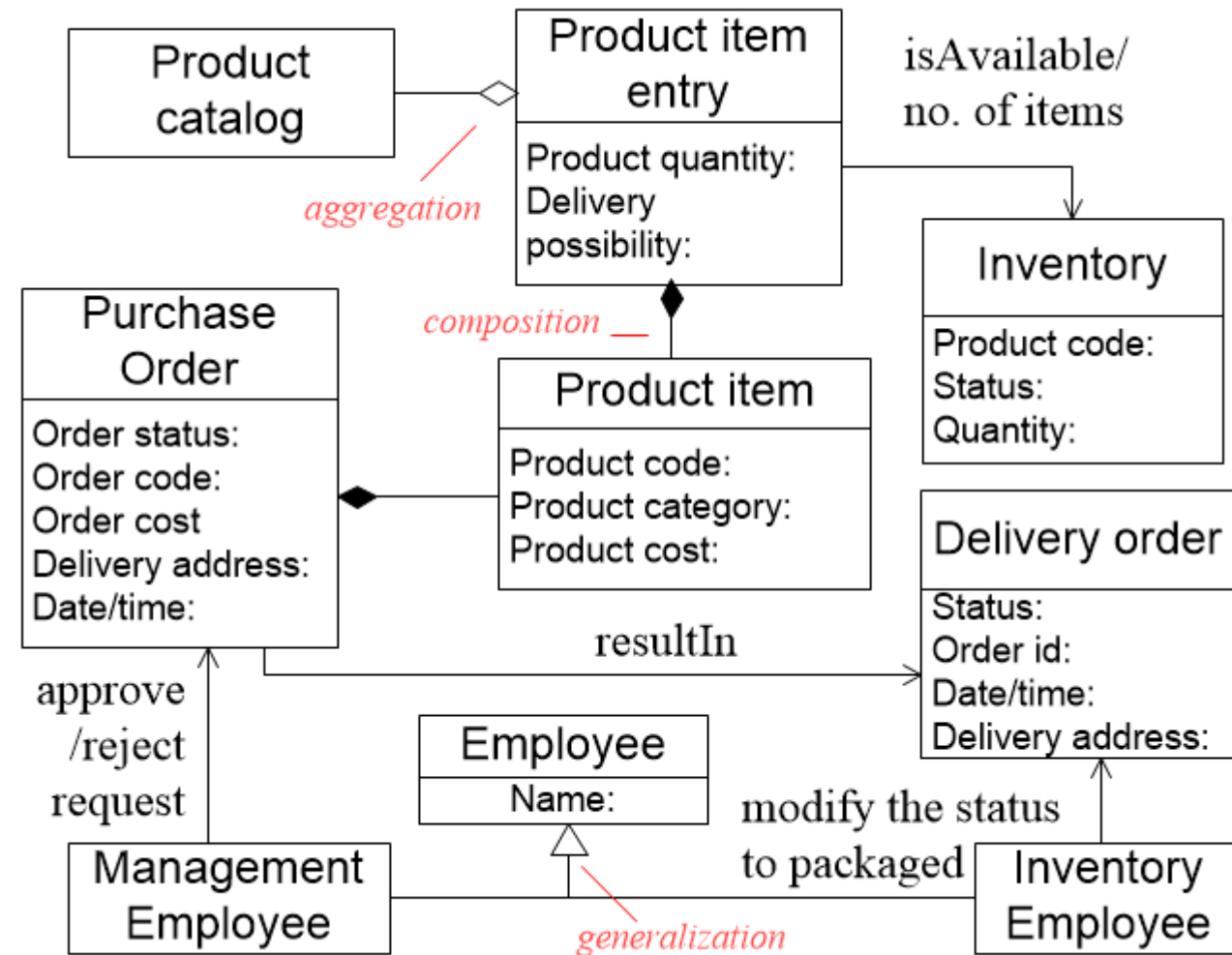


Aggregation implies a relationship where a class can exist independently of the other class. **E.g.**, If we delete the product catalog, the product item entry still exist.

Composition implies a relationship where the class cannot exist independent of the other class. **E.g.**, If we delete the product item entry, the product item should not exist.

Class diagrams – UML

A class diagrams can be used to describe a system in terms of the entities “composing” it and their inter-relationships.

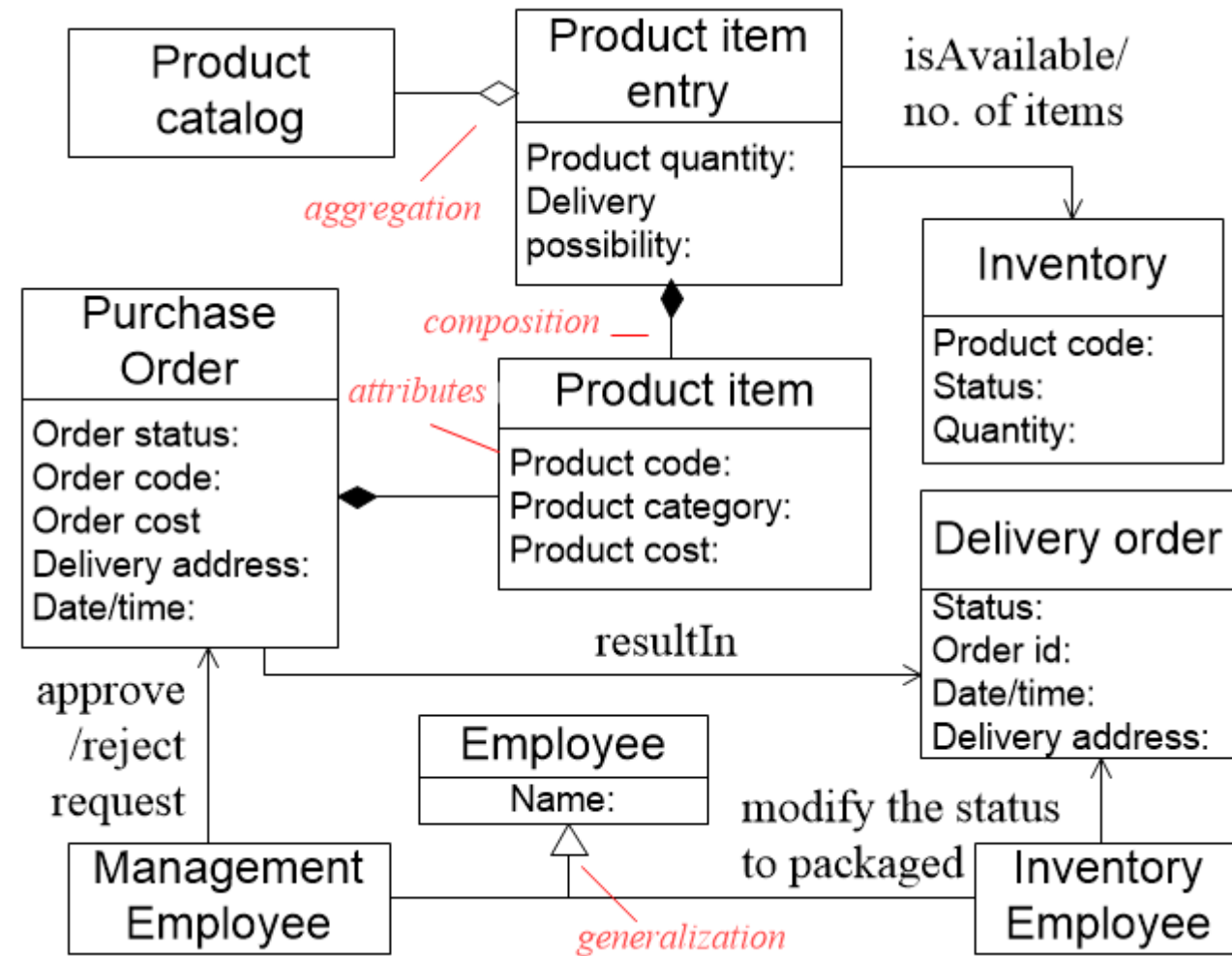


Generalization is a mechanism for combining similar classes of objects into a single, more general class.

Specialization is the reverse process of Generalization that means creating new sub classes from an existing more general class.

Class diagrams – UML

A class diagrams can be used to describe a system in terms of the entities “composing” it and their inter-relationships.

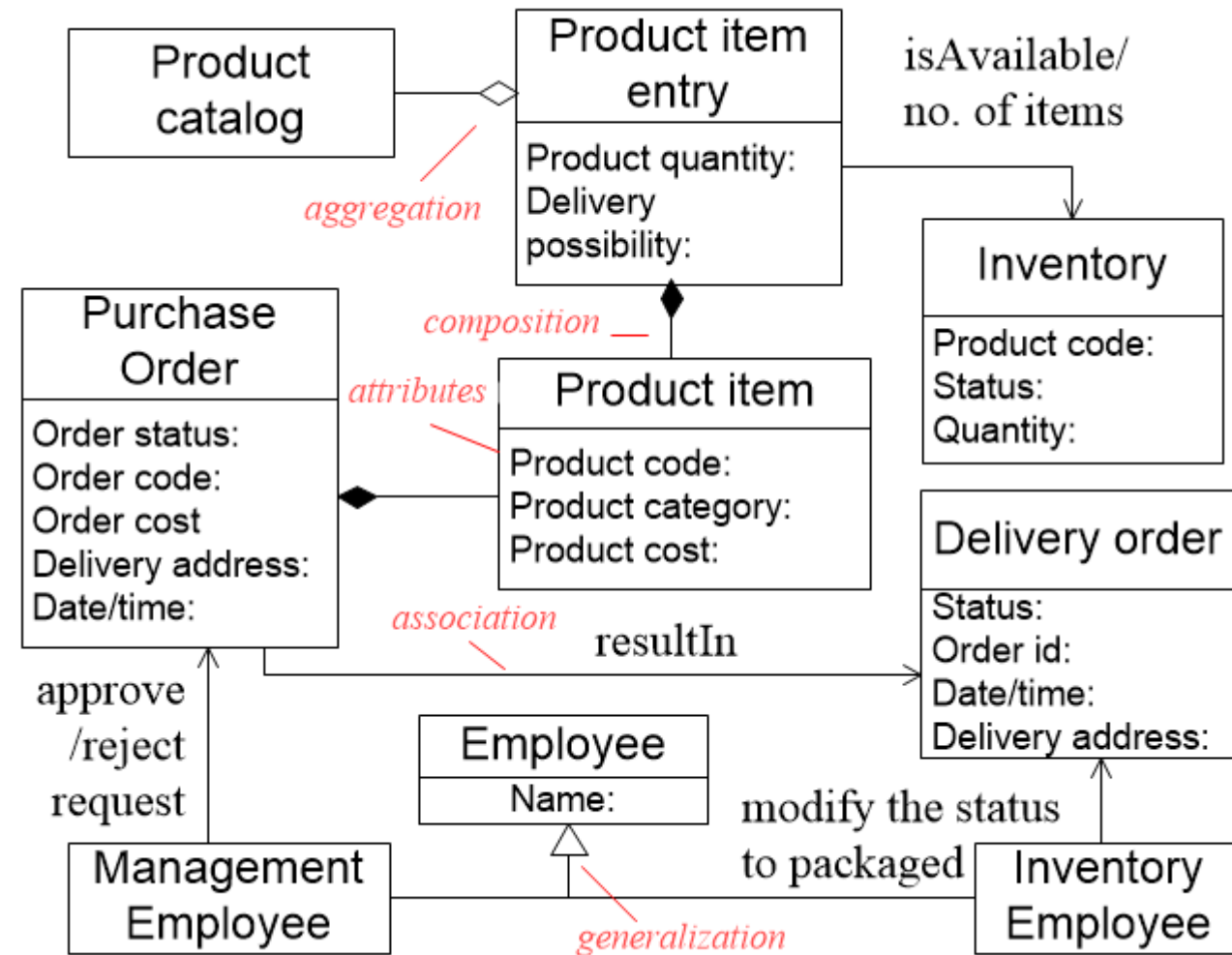


An attribute (property) is a named slot within a classifier that describes a range of values that instances of the classifier may hold.

Attributes are always single-valued in UML, and their value might be derived.

Class diagrams – UML

A class diagrams can be used to describe a system in terms of the entities “composing” it and their inter-relationships.

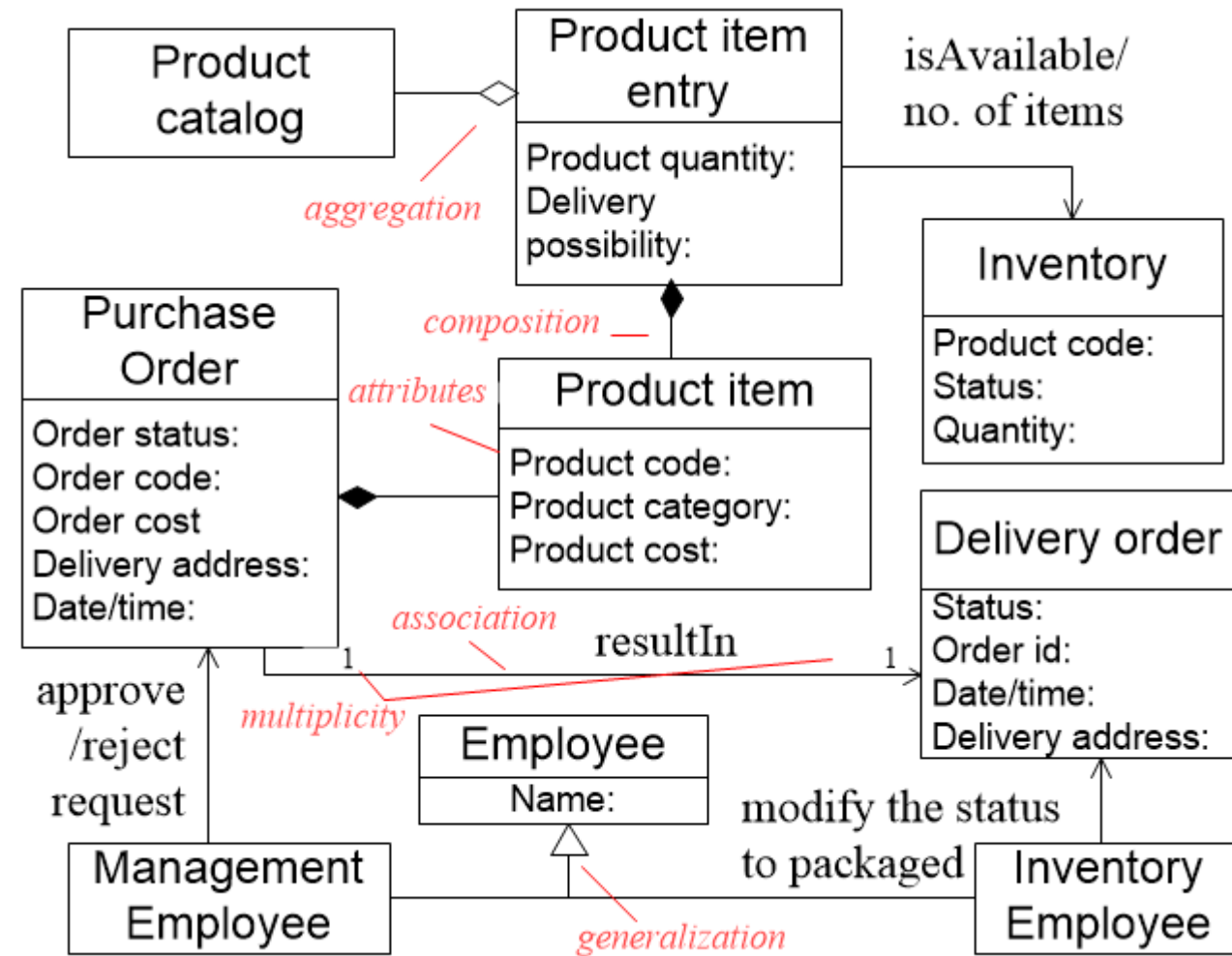


Associations represent semantic relationships. Each association can have up to two roles for participating objects.

Associations represented with directed arrows are navigable only in one direction.

Class diagrams – UML

A class diagrams can be used to describe a system in terms of the entities “composing” it and their inter-relationships.



Associations represent semantic relationships. Each association can have up to two roles for participating objects.

Each role can also have an associated cardinality range (“multiplicity”).

Multiplicity allows to specify cardinality, i.e., number of elements.

Part 5

Domain models and domain modeling

Domain models

- A domain model is a **conceptual model** for a **specific domain**.*
- A domain model aims at defining the **key concepts** of a specific **domain** as well as various **relationships** among these **concepts**.*
- A domain model uses high-level non-technical terms/concepts (e.g., almost every thing you can imagine).*

A domain model is not a model of the software.

Domain models

A *concept* is an entity, a thing, an object or even an idea in the domain.

A *domain concept* can be represented as a ***conceptual class***.

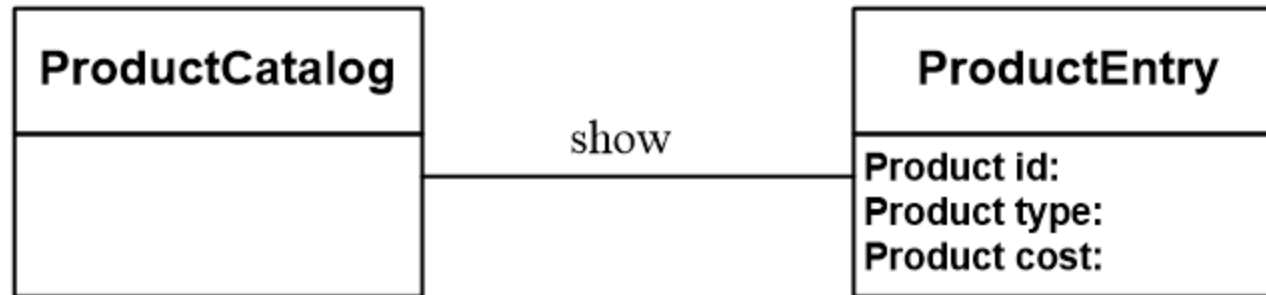
A *domain concept/class* is **not a software object/class** (e.g., Java classes).

This means, **no operations** can be in ***domain models***, which only **include**:

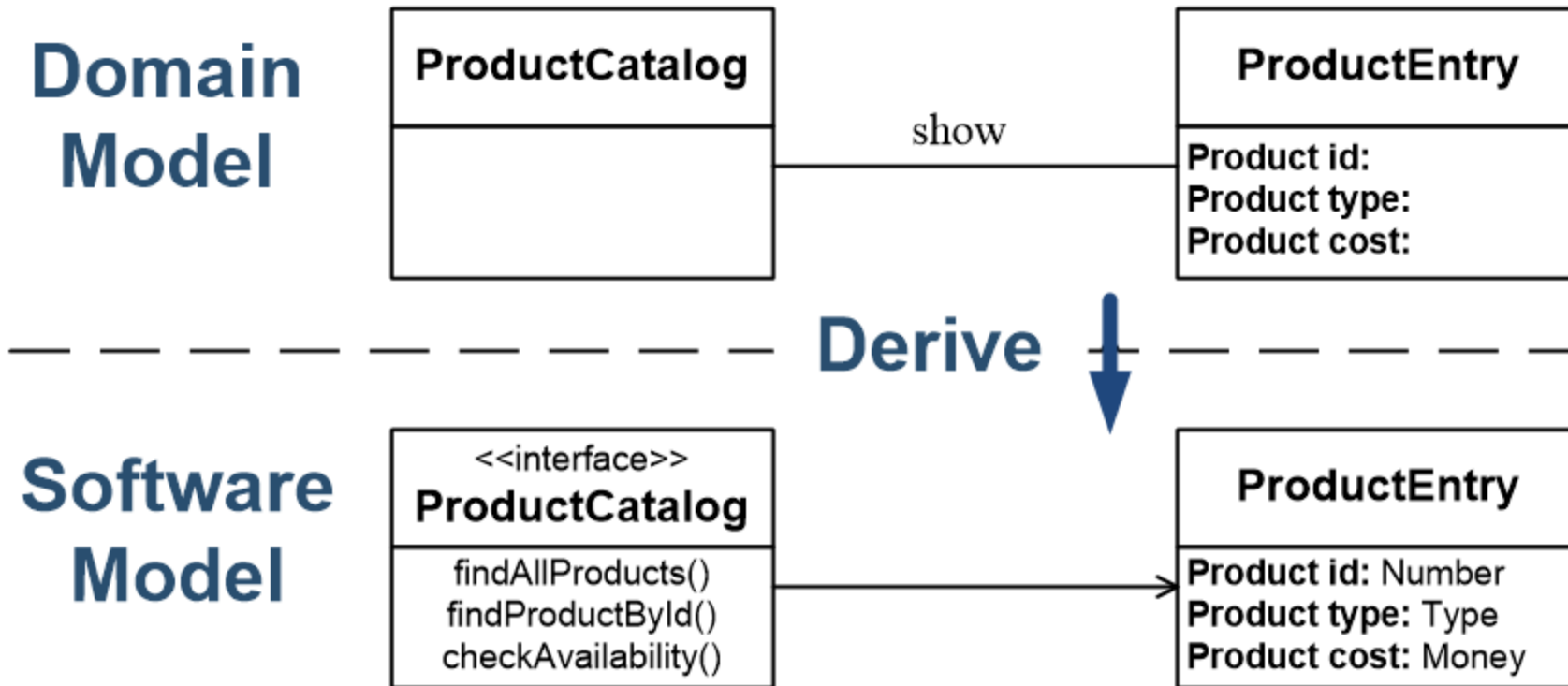
- ***conceptual classes*** and their ***attributes***.
- ***relationships/associations*** between these ***classes***.

Domain models vs Software models

**Domain
Model**



Domain models vs Software models



Domain modeling

A **domain model** should include a set of **fundamental language constructs** that represent the key **concepts** of the domain along with the **relationships** among them as well as a set of **constraints** that govern how such constructs can be used to produce valid models*.

The development of a **domain model** required several **iterations**, and came to its end when the model **is considered complete** with respect to the main properties of the domain that **we aim to capture**.

* A systematic approach to domain-specific language design using UML. In: 10th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing, ISORC 2007, IEEE, pp 2–9



Developing a domain modeling

Developing a domain modeling

Identifying concepts:

Analyze the *domain of interest* and try to identify *key terms*.

Usually, not all *key terms* are *concepts* but only a *subset* of them.

If the system requirements are available, they can be a very good source for identifying key concepts.

How?

Start by identifying all nouns and, then, determine which of them can be considered suitable concepts.

Concepts identification – an example

- R1.** The system should show products, their prices, etc. in a product catalogue.
- R2.** The system should allow a customer to check the availability of a given product.
- R3.** The system should allow a customer to check whether the product can be delivered to a specific location.
- R4.** The system should allow a customer to purchase a product by completing a purchase order.
- R5.** The system should allow a customer to modify/cancel its order within 24 hours.
- R6.** The system should allow a customer to request modifying/cancelling its order within 24-48 hours. Such request can be approve/reject by an employee.
- R7.** The system should issue an invoice for each purchase order.
- R8.** The system should allow a customer to pay for her purchase invoice by various means.
- R9.** A successful purchase order result in a delivery order.
- R10.** An inventory employee can change the status of a delivery order.

Concepts identification – an example

- R1.** The system should show products, their prices, etc. in a product catalogue.
- R2.** The system should allow a customer to check the availability of a given product.
- R3.** The system should allow a customer to check whether the product can be delivered to a specific location.
- R4.** The system should allow a customer to purchase a product by completing a purchase order.
- R5.** The system should allow a customer to modify/cancel its order within 24 hours.
- R6.** The system should allow a customer to request modifying/cancelling its order within 24-48 hours. Such request can be approve/reject by an employee.
- R7.** The system should issue an invoice for each purchase order.
- R8.** The system should allow a customer to pay for her purchase invoice by various means.
- R9.** A successful purchase order result in a delivery order.
- R10.** An inventory employee can change the status of a delivery order.

Concepts identification – an example

- R1. The system should show products, their prices, etc. in a product catalogue.
- R2. The system should allow a customer to check the availability of a given product.
- R3. The system should allow a customer to check whether the product can be delivered to a specific location.
- R4. The system should allow a customer to purchase a product by completing a purchase order.
- R5. The system should allow a customer to modify/cancel its order within 24 hours.
- R6. The system should allow a customer to request modifying/cancelling its order within 24-48 hours. Such request can be approve/reject by an employee.
- R7. The system should issue an invoice for each purchase order.
- R8. The system should allow a customer to pay for her purchase invoice by various means.
- R9. A successful purchase order result in a delivery order.
- R10. An inventory employee can change the status of a delivery order.

Customer

Orders management
Employee

Inventory
Employee

Customer

Product
catalog

Order Request

Payment

Purchase Order

Product item

Delivery order

invoice

Orders management
Employee

Inventory
Employee

Customer

Product
catalog

Order Request

Product item
entry

Inventory

Payment

Purchase Order

Product item

Delivery order

invoice

Orders management
Employee

Inventory
Employee



Developing a domain modeling

Identifying attributes:

- Attributes describe the properties of concepts.
- Attributes are needed to store information that an object may use.

Most attributes can be identified by asking:

- What status the concept can be in?
- What information do the concept need to store?

Attributes identification – an example

- R1. The system should show products, their prices, etc. in a product catalogue.
- R2. The system should allow a customer to check the availability of a given product.
- R3. The system should allow a customer to check whether the product can be delivered to a specific location.
- R4. The system should allow a customer to purchase a product by completing a purchase order.
- R5. The system should allow a customer to modify/cancel its order within 24 hours.
- R6. The system should allow a customer to request modifying/cancelling its order within 24-48 hours. Such request can be approve/reject by an employee.
- R7. The system should issue an invoice for each purchase order.
- R8. The system should allow a customer to pay for her purchase invoice by various means.
- R9. A successful purchase order result in a delivery order.
- R10. An inventory employee can change the status of a delivery order.

Attributes identification – an example

- R1. The system should show products, their prices, etc. in a product catalogue.
- R2. The system should allow a customer to check the availability of a given product.
- R3. The system should allow a customer to check whether the product can be delivered to a specific location.
- R4. The system should allow a customer to purchase a product by completing a purchase order.
- R5. The system should allow a customer to modify/cancel its order within 24 hours.
- R6. The system should allow a customer to request modifying/cancelling its order within 24-48 hours. Such request can be approve/reject by an employee.
- R7. The system should issue an invoice for each purchase order.
- R8. The system should allow a customer to pay for her purchase invoice by various means.
- R9. A successful purchase order result in a delivery order.
- R10. An inventory employee can change the status of a delivery order.

Customer

Product
catalog

Order Request

Product item
entry

Inventory

Payment

Purchase Order

Product item

Delivery order

invoice

Employee

Orders management
Employee

Inventory
Employee



Customer

Product
catalog

Order Request

Product code:
Product category:
Product cost:
Order: cost
Delivery address:
Date/time:

Product item
entry

Product code:
Product category:
Product cost:
Product quantity:
Delivery
possibility:

Inventory

Product code:
Status:
Quantity:

Payment

Status:
Payment code:
Method:
Invoice code:

Purchase Order

Order status:
Order code:
Product code:
Product category:
Product cost:
Order cost
Delivery address:
Date/time:

Product item

Product code:
Product category:
Product cost:

Delivery order

Status:
Order id:
Date/time:
Delivery address:

invoice

Invoice code:
Order code:
Invoice status:
Payment method:
Date/time of
amount:

Employee

Name:

Orders management
Employee

Inventory
Employee



A concept or an attribute

An attributes describe the properties of concepts.

An attributes is needed to store information that an object may use.

An attribute should have values of a primitive type, e.g. integer, string, Boolean, enumeration, etc.

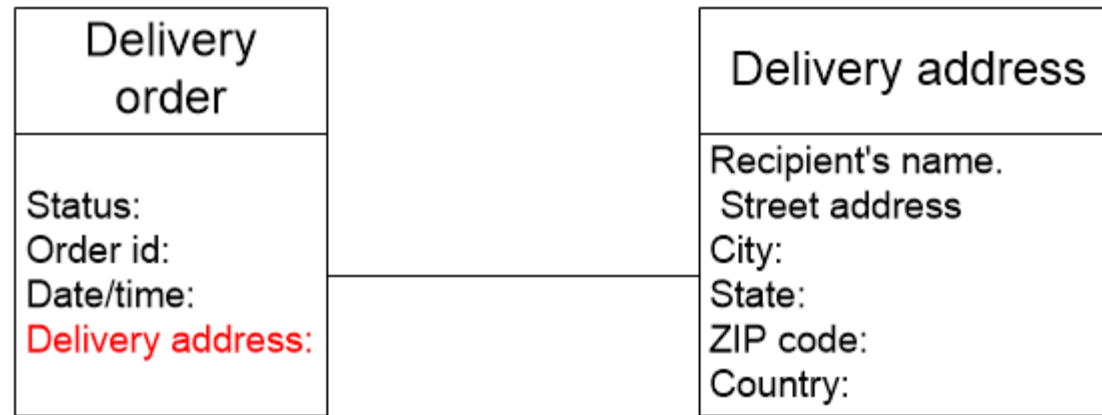
A concept or an attribute

Order id is a potential attribute, but what about Delivery address?

Delivery order
Status: Order id: Date/time: Delivery address:

A concept or an attribute

Order id is a potential attribute, but what about Delivery address?



If Delivery address is represented as a **primitive type** (e.g., a string), it can be an **attribute**.

If Delivery address is not a **primitive type**, it should be considered a **concept**.

Customer

Product
catalog

Order Request

Product code:
Product category:
Product cost:
Order: cost
Delivery address:
Date/time:

Product item
entry

Product code:
Product category:
Product cost:
Product quantity:
Delivery
possibility:

Inventory

Product code:
Status:
Quantity:

Payment

Status:
Payment code:
Method:
Invoice code:

Purchase Order

Order status:
Order code:
Product code:
Product category:
Product cost:
Order cost
Delivery address:
Date/time:

Product item

Product code:
Product category:
Product cost:

Delivery order

Status:
Order id:
Date/time:
Delivery address:

Delivery address

Recipient's name.
Street address
City:
State:
ZIP code:
Country:

invoice

Invoice code:
Order code:
Invoice status:
Payment method:
Date/time of
amount:

Employee

Name:

Orders management
Employee

Inventory
Employee



Developing a domain modeling

Identifying relationships:

When you define a *relationship*, you should keep in mind that such *relationship* should hold between the two *classes* all the time.

Relationships identification

- R1. The system should show products, their prices, etc. in a product catalogue.
- R2. The system should allow a customer to check the availability of a given product.
- R3. The system should allow a customer to check whether the product can be delivered to a specific location.
- R4. The system should allow a customer to purchase a product by completing a purchase order.
- R5. The system should allow a customer to modify/cancel its order within 24 hours.
- R6. The system should allow a customer to request modifying/cancelling its order within 24-48 hours. Such request can be approve/reject by an employee.
- R7. The system should issue an invoice for each purchase order.
- R8. The system should allow a customer to pay for her purchase invoice by various means.
- R9. A successful purchase order result in a delivery order.
- R10. An inventory employee can change the status of a delivery order.

Relationships identification

- R1. The system should show products, their prices, etc. in a product catalogue.
- R2. The system should allow a customer to check the availability of a given product.
- R3. The system should allow a customer to check whether the product can be delivered to a specific location.
- R4. The system should allow a customer to purchase a product by completing a purchase order.
- R5. The system should allow a customer to modify/cancel its order within 24 hours.
- R6. The system should allow a customer to request modifying/cancelling its order within 24-48 hours. Such request can be approve/reject by an employee.
- R7. The system should issue an invoice for each purchase order.
- R8. The system should allow a customer to pay for her purchase invoice by various means.
- R9. A successful purchase order result in a delivery order.
- R10. An inventory employee can change the status of a delivery order.

Customer

Product
catalog

Order Request

Product code:
Product category:
Product cost:
Order: cost
Delivery address:
Date/time:

Product item
entry

Product code:
Product category:
Product cost:
Product quantity:
Delivery
possibility:

Inventory

Product code:
Status:
Quantity:

Payment

Status:
Payment code:
Method:
Invoice code:

Purchase Order

Order status:
Order code:
Product code:
Product category:
Product cost:
Order cost
Delivery address:
Date/time:

Product item

Product code:
Product category:
Product cost:

Delivery order

Status:
Order id:
Date/time:
Delivery address:

Delivery address

Recipient's name.
Street address
City:
State:
ZIP code:
Country:

invoice

Invoice code:
Order code:
Invoice status:
Payment method:
Date/time of
amount:

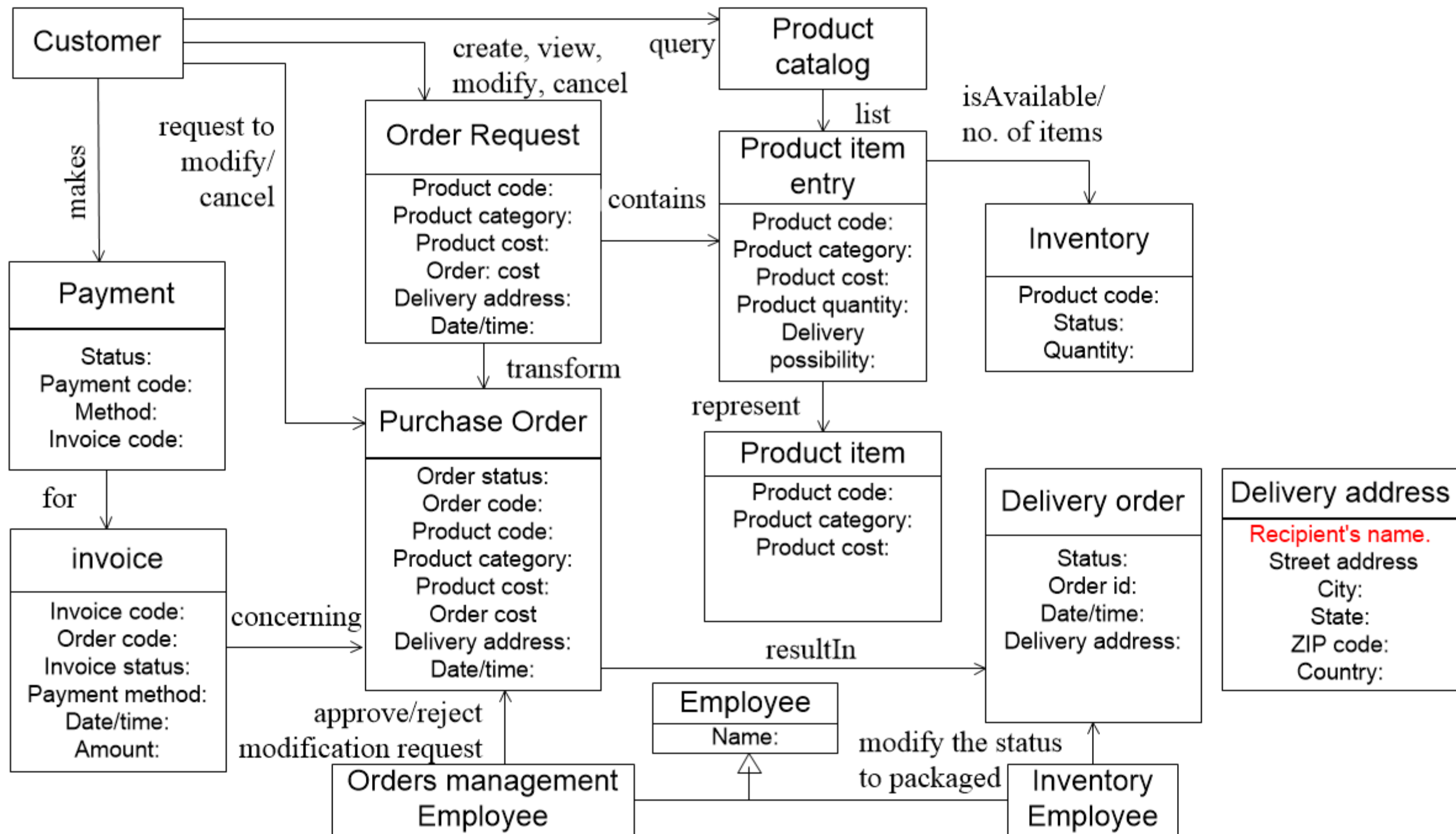
Employee

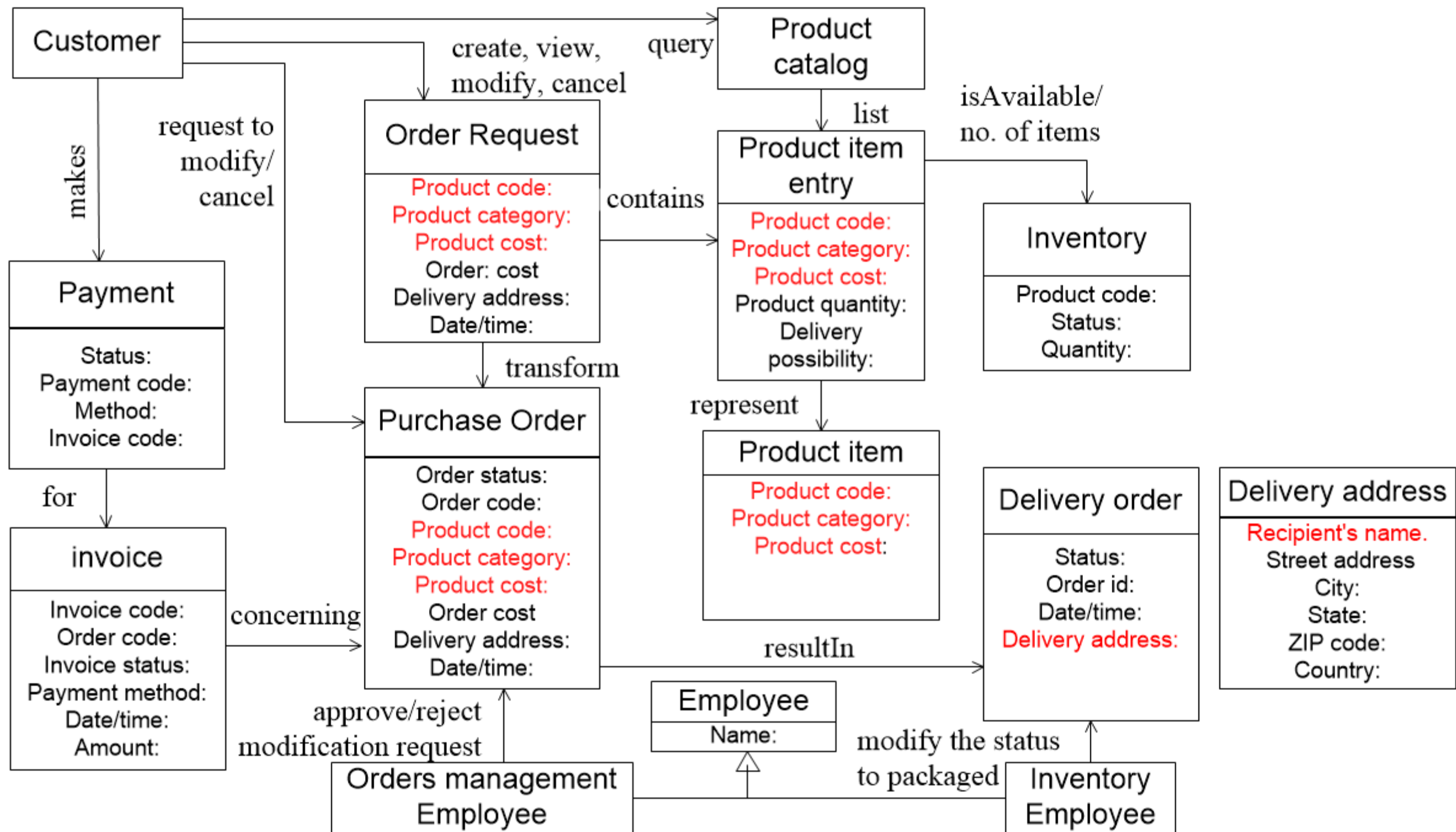
Name:

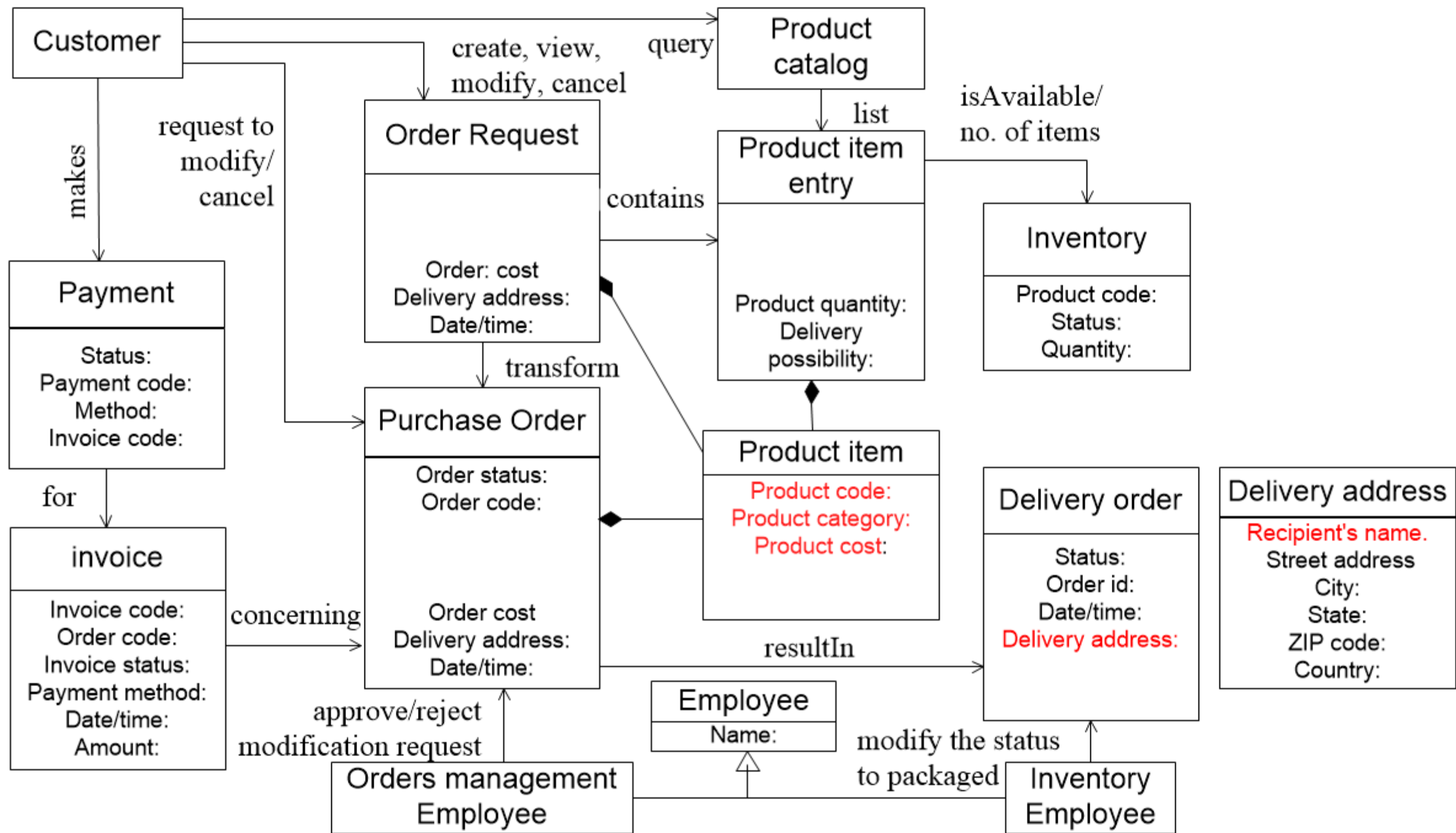
Orders management
Employee

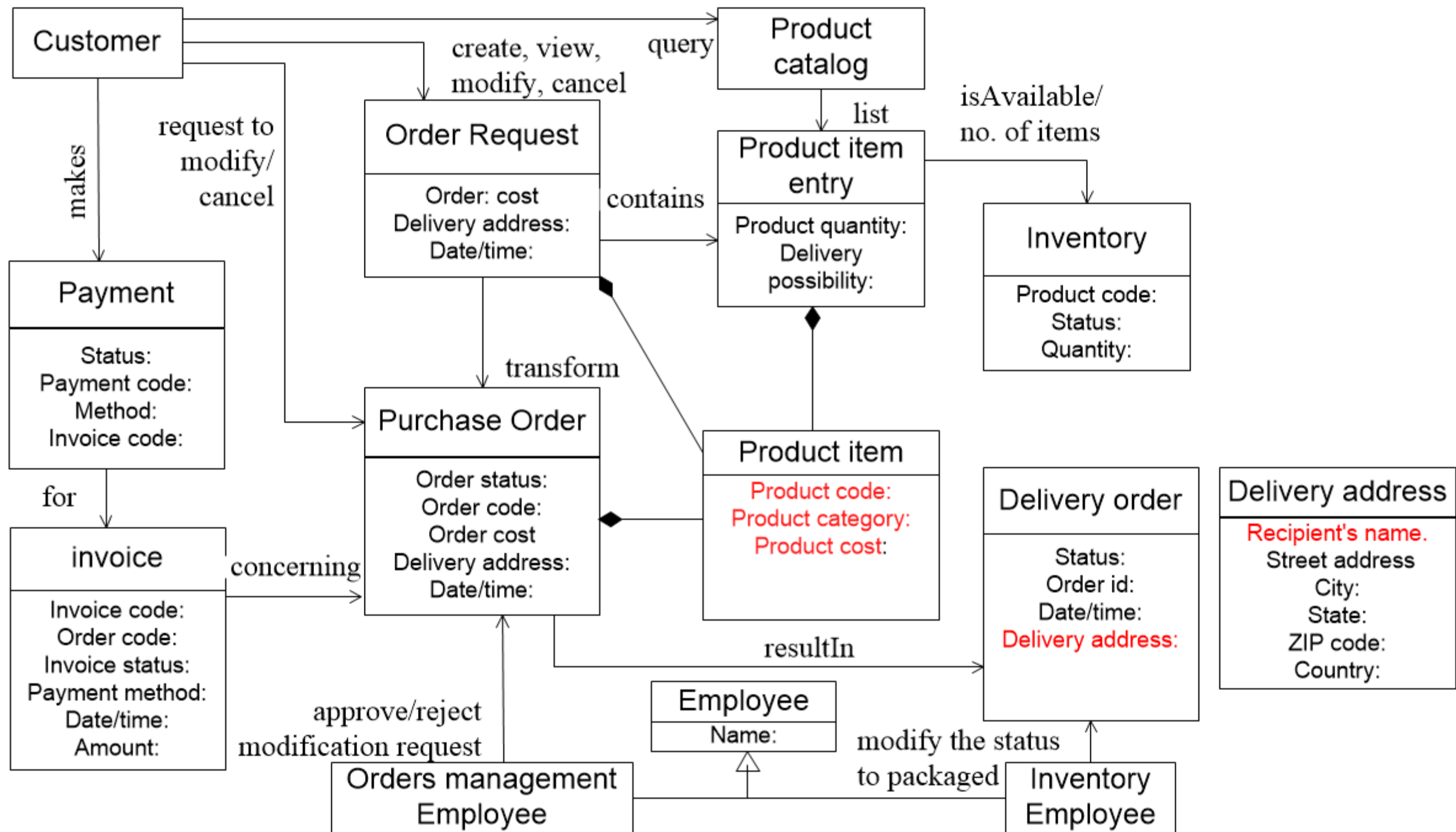
Inventory
Employee

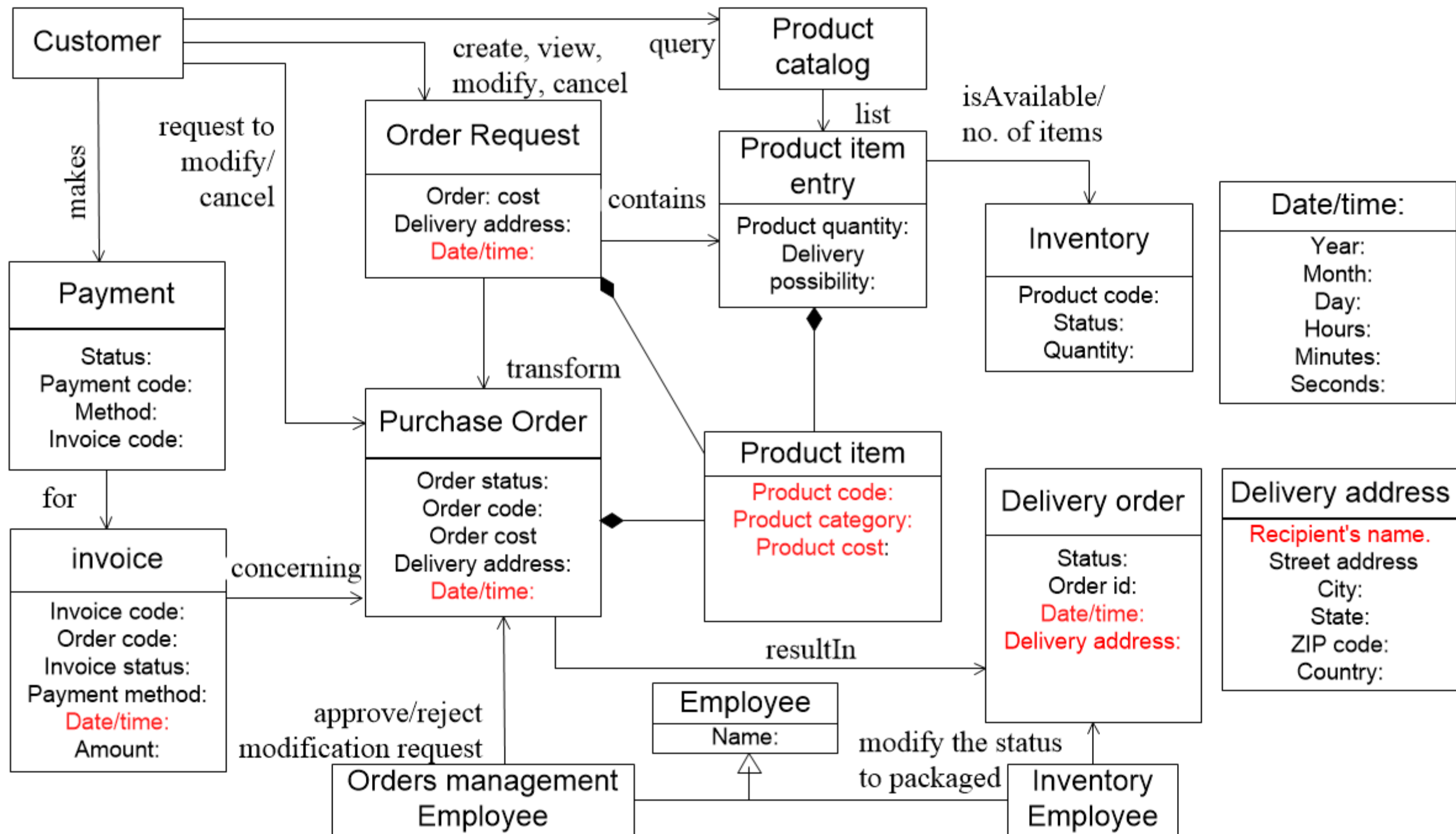


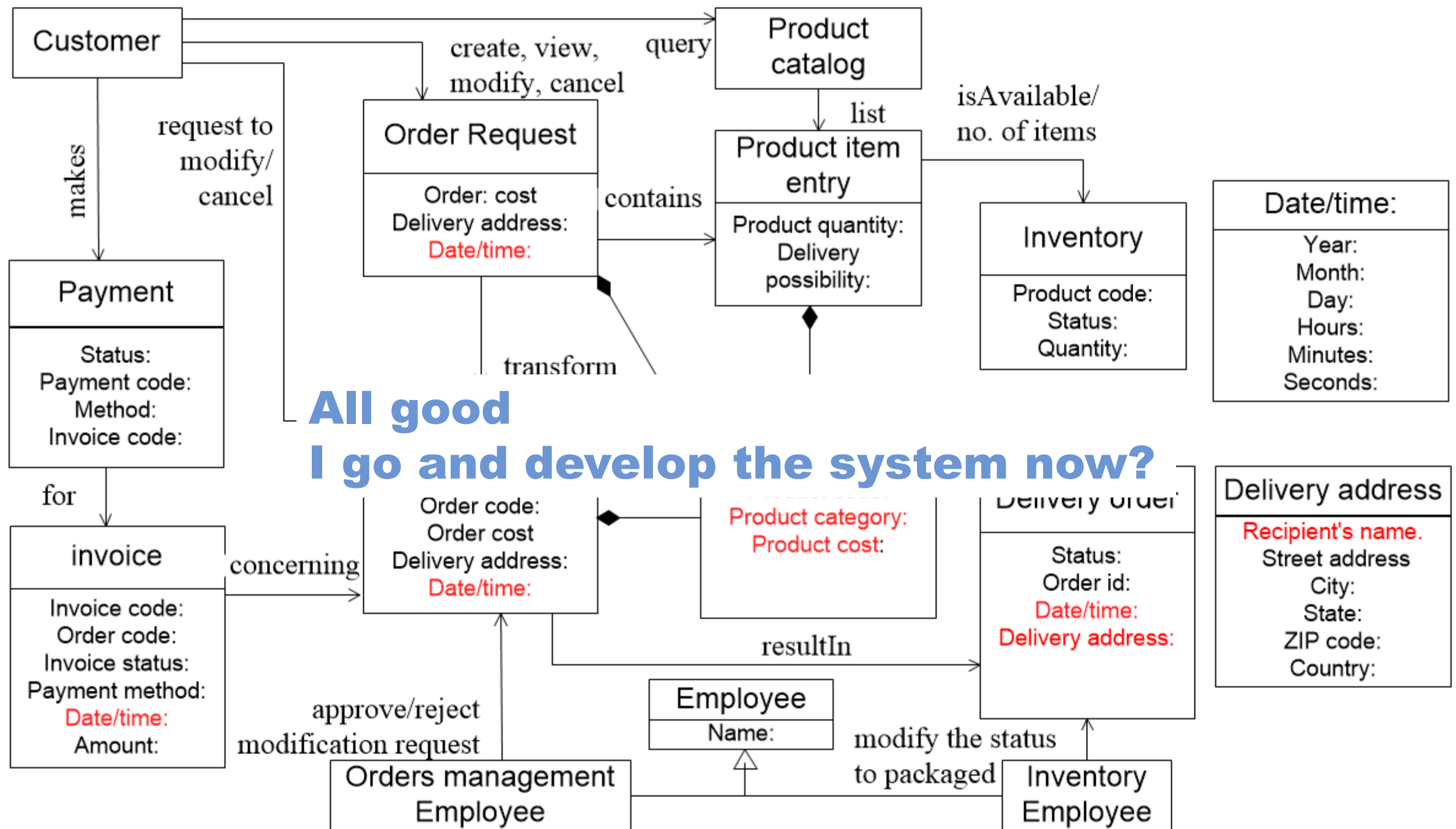


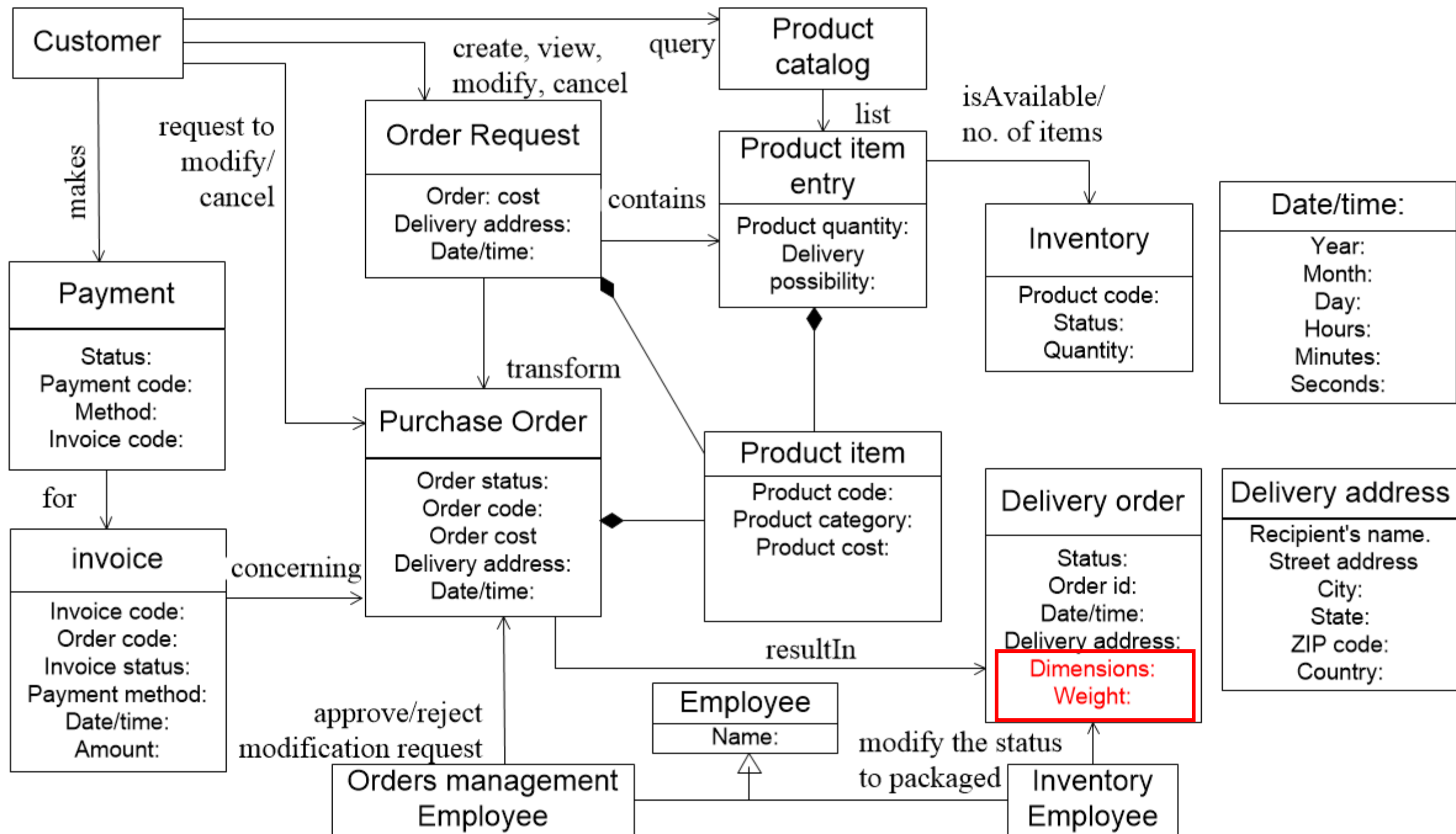


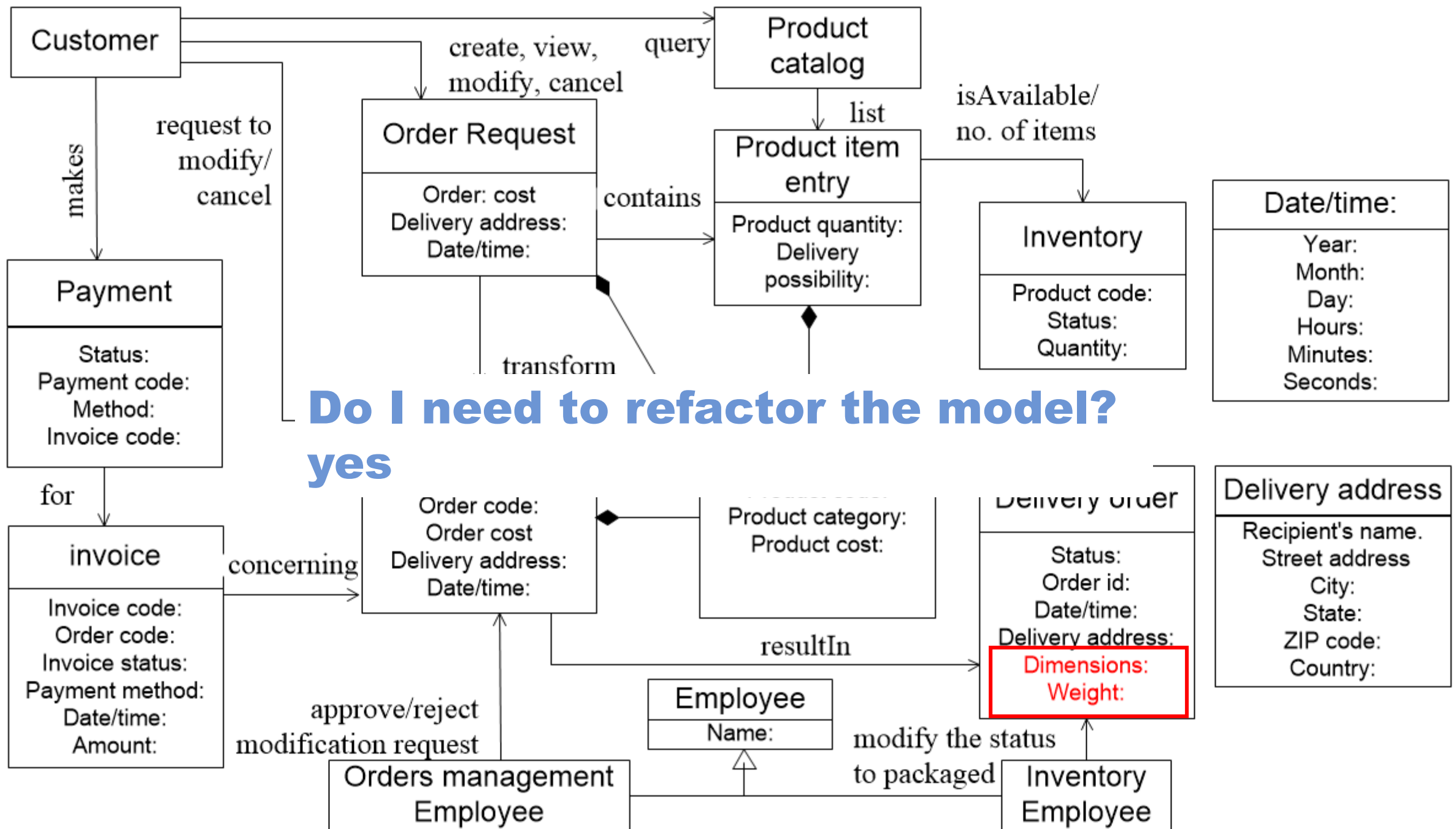


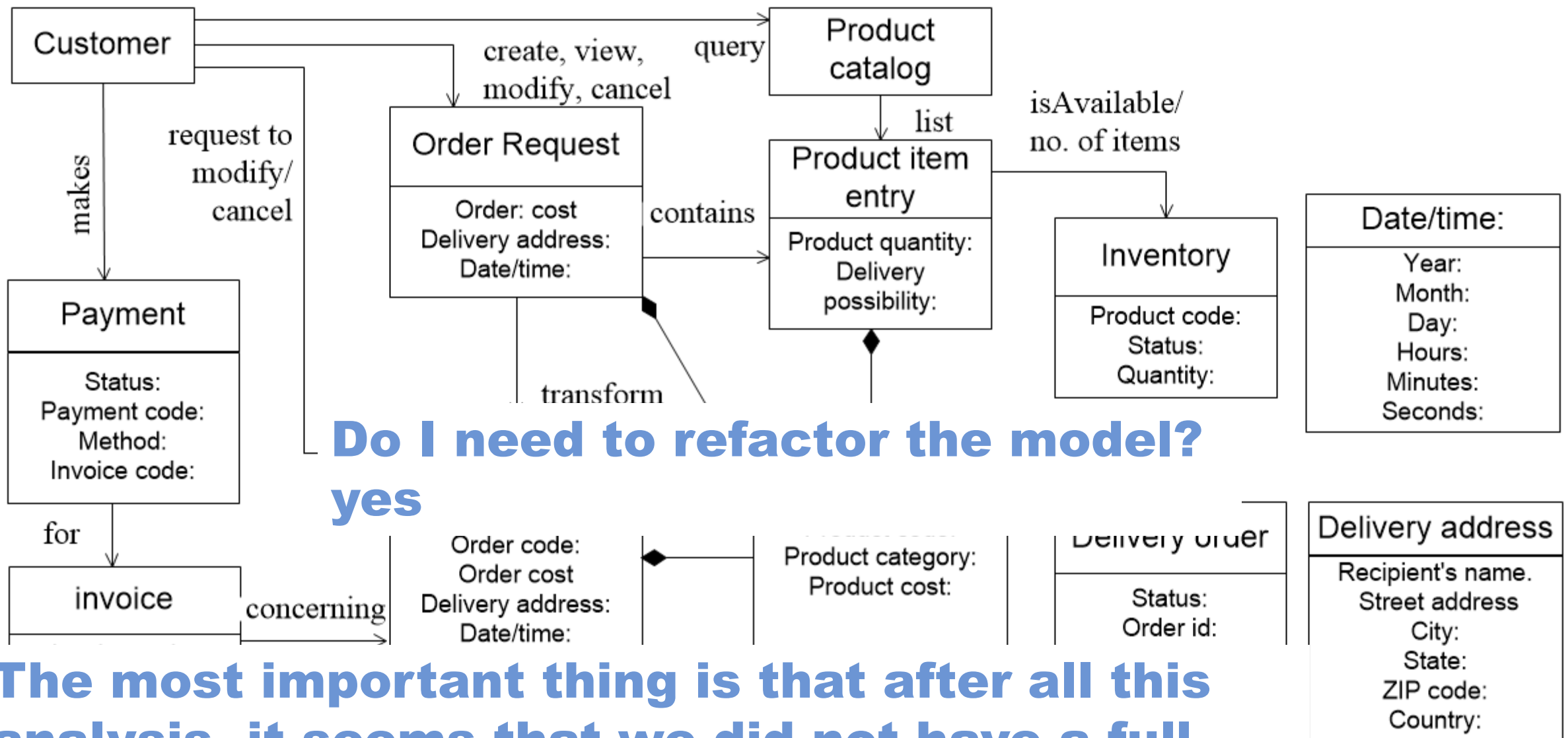












Do I need to refactor the model?
yes

The most important thing is that after all this analysis, it seems that we did not have a full understanding of what a product is!!



UNIVERSITY OF TARTU

**Thank You
for your attention**

Mohamad Gharib
mohamad.gharib@ut.ee



unitartu



tartuuniversity

